NATIONAL ECONOMICS UNIVERSITY
SCHOOL OF TECHNOLOGY



REPORT

STATISTICAL LEARNING

Project title:

PREDICTING LOAN DEFAULT ON LENDING CLUB DATA

Group **[no.]**: **[STUDENT NAME 1 – ID]**
[STUDENT NAME 2 – ID]
Class **[DS66A / DS66B]**
Supervisor: **[Supervisor's full name]**

Hanoi, **[month year]**

TABLE OF CONTENTS

LIST OF SYMBOLS AND ABBREVIATIONS i

LIST OF FIGURES iii

LIST OF TABLES v

1 OVERVIEW OF THE PROBLEM 1

1.1 Credit-risk scoring and peer-to-peer lending 1

1.2 Challenges 1

1.3 Motivation, objectives and approach 2

1.4 Chapter summary 3

2 PROBLEM FORMULATION AND THEORETICAL BACKGROUND 4

2.1 Problem formulation 4

2.2 Related work 4

2.2.1 Statistical credit scoring 4

2.2.2 Studies on Lending Club data 5

2.2.3 Open-source projects 5

2.2.4 Positioning and gap 6

2.3 Models 6

2.3.1 Logistic regression with L2 regularisation 6

2.3.2 Decision tree 6

2.3.3 Random forest 7

2.3.4 Histogram gradient boosting 7

2.4 Evaluation metrics and statistical tools 7

2.5 Chapter summary 8

3 DATA AND METHODOLOGY 9

3.1 Dataset and data dictionary . . . 9
3.2 Label and population . . . 9
3.3 Out-of-time split . . . 10
3.4 Leakage controls . . . 10
3.5 Cleaning rules . . . 11
3.6 Column grouping and feature selection . . . 11
3.7 Feature engineering . . . 12
3.8 Preprocessing pipeline . . . 12
3.9 Models and tuning protocol . . . 12
3.10 Evaluation protocol . . . 13
3.11 Follow-up experiments . . . 13
3.12 Implementation and reproducibility . . . 14
3.13 Chapter summary . . . 14

4 EXPERIMENTS AND RESULTS 15

4.1 Experimental setup . . . 15
4.2 Dataset statistics and the time structure . . . 15
4.3 Evidence for the leakage controls and the column groups . . . 20
4.4 Cleaning checks, EDA and feature selection . . . 22
4.5 Evidence for the preprocessing choices . . . 27
4.6 Model comparison . . . 29
4.6.1 Hyperparameter tuning . . . 29
4.6.2 Validation performance and overfitting check . . . 30
4.6.3 Bootstrap confidence intervals . . . 32
4.6.4 Final evaluation on the test set . . . 32
4.6.5 Logistic coefficients . . . 33
4.7 Follow-up experiments . . . 33
4.7.1 Experiment 1: removing `sub_grade` . . . 33
4.7.2 Experiment 2: compact 12-field scorecard . . . 35
4.8 Discussion . . . 37

4.9 Limitations and threats to validity 38
4.10 Chapter summary 39

CONCLUSION AND FUTURE WORK **40**

REFERENCES **42**

APPENDIX **44**

LIST OF SYMBOLS AND ABBREVIATIONS

Abbreviation	Meaning
AUC	Area Under the (ROC) Curve
B (bootstrap)	Number of bootstrap resamples ($B = 1{,}000$)
CI	Confidence Interval
CV	Cross-Validation
DTI	Debt-To-Income ratio
EDA	Exploratory Data Analysis
FICO	Credit score issued by the Fair Isaac Corporation
FPR	False Positive Rate
Gini	Gini coefficient, $2\,\mathrm{AUC} - 1$
HistGB	Histogram-based Gradient Boosting (`HistGradientBoostingClassifier`)
KS	Kolmogorov–Smirnov statistic, $\max(\mathrm{TPR} - \mathrm{FPR})$
LC	Lending Club
LR	Logistic Regression
OOT	Out-Of-Time (temporal hold-out)
P2P	Peer-to-Peer (lending)
PDO	Points to Double the Odds (scorecard scaling)
RF	Random Forest
ROC	Receiver Operating Characteristic
TPR	True Positive Rate
y_i	Label of loan i: 1 = Charged Off, 0 = Fully Paid
$\mathbf{x}_i$	Application-time feature vector of loan i
p_i	Predicted probability (risk score) that loan i defaults
C	Inverse regularisation strength of logistic regression

LIST OF FIGURES

Figure 4.1 Left: number of labelled loans per issue year up to 2015, with the pre-2012 share. Right: number of numeric columns that first have data in each year, with the cumulative count. 17

Figure 4.2 Left: label balance in the training set. Right: default rate in each time-based set. 18

Figure 4.3 Number of loans (bars) and default rate (line) by issue month, 2012–2015, with the two cut dates separating train, validation and test. . 19

Figure 4.4 Share of missing values for each of the 151 raw columns in the training set (sorted; names hidden). The dashed line marks the 30% drop threshold. 20

Figure 4.5 Categorisation of the 151 raw columns into rule groups; green marks the 57 candidate columns. 21

Figure 4.6 Default rate by loan purpose in the training set (dashed line: overall 13.7%). 23

Figure 4.7 Default rate by sub-grade (A1–G5) in the training set. 24

Figure 4.8 Distribution of the interest rate by grade in the training set; rates step up with the grade. 24

Figure 4.9 Default rate by decile of six numeric features in the training set (dashed line: overall default rate). Titles give the median value of the first and last decile; matplotlib renders the two dollar signs in some titles as mathematical text. 25

Figure 4.10 Box plots (outliers hidden) of `fico_avg` and `loan_to_income` for repaid and defaulted loans in the training set. 25

Figure 4.11 Top 15 numeric candidates by single-column ROC-AUC in the training set. 26

Figure 4.12 Pearson correlation matrix of eight key numeric features in the training set; `loan_amnt` and `installment` have $r = 0.994$. 27

Figure 4.13 Left: skewness of the most skewed numeric features with missing data. Right: mean-to-median ratio for the same features. 28

Figure 4.14 Distribution of `total_rev_hi_lim` (below the 99th percentile): raw data, after median imputation and after mean imputation. 28

Figure 4.15 Raw scale of seven financial features (logarithmic axis) in a sample of 10,000 training loans. 29

Figure 4.16 The same features after standardisation. 29

Figure 4.17 Validation set. Left: ROC curves with AUC and Gini of the five models. Right: ROC-AUC, Gini and KS of each model. 31

Figure 4.18 Cumulative separation (KS) charts on the validation set for the baseline (left) and logistic regression (right). 32

Figure 4.19 Validation ROC curves (left) and AUC/Gini/KS bars (right) for the baseline, logistic without `sub_grade` and logistic with all features. . 34

Figure 4.20 Left: coefficients of eight numeric features with and without `sub_grade`. Right: all 118 coefficients present in both models (dashed line: unchanged). 35

Figure 4.21 Left: density of the scorecard points for repaid and defaulted test loans. Right: weights of the ten numeric features in the compact scorecard. 37

LIST OF TABLES

Table 3.1 Models and hyperparameter grids searched with `TimeSeriesSplit(3)` inside the training set. 13

Table 4.1 Loan statuses in the raw file and the share of loans with outstanding principal (`out_prncp` > 0). 16

Table 4.2 Loans per issue year: share with a final label, default rate among labelled loans and share of 60-month loans. 16

Table 4.3 Check of the matured-loan rule. 17

Table 4.4 Filtering funnel. 18

Table 4.5 Train, validation and test sets. 19

Table 4.6 Suspect columns with the highest single-column ROC-AUC on the training set (best application-time column: `sub_grade`, 0.6565). . . . 21

Table 4.7 Number of the 151 raw columns in each rule group. 22

Table 4.8 Rows violating each logic rule before cleaning (after cleaning, only `revol_util` > 100 remains because it is a valid case). 23

Table 4.9 Numeric column pairs with $|r| \geq 0.90$ in the training set and the column dropped. 26

Table 4.10 Selected hyperparameters and cross-validated ROC-AUC (`TimeSeriesSplit(3)` inside the training set). 30

Table 4.11 Model comparison on the validation set (92,631 loans) and train–validation AUC gap. 31

Table 4.12 Paired-bootstrap difference in validation AUC against the `sub_grade` baseline ($B = 1{,}000$). 32

Table 4.13 Single evaluation on the test set (88,669 loans, 2015-10 to 2015-12). 33

Table 4.14 Effect of removing `sub_grade` (logistic regression, $C = 1$ for both variants). 34

Table 4.15 Standardised logistic coefficients of eight numeric features with and without `sub_grade`. 35

Table 4.16 Compact 12-field logistic model versus the full model and the baseline. 36

Table 4.17 Score bands of the compact scorecard: share of loans and observed default rate (cut-offs chosen on validation, applied unchanged to test). . 36

Table 4.18 Input features of the final pipeline. 44

CHAPTER 1. OVERVIEW OF THE PROBLEM

This chapter introduces credit-risk scoring on a peer-to-peer platform, explains why the problem is harder than it looks, and states the objectives and the approach followed in the rest of the report.

1.1 Credit-risk scoring and peer-to-peer lending

Credit scoring is the practice of turning information about an applicant into a number that orders applicants by their risk of not repaying; lenders use that number to accept or reject applications, to price loans and to set limits. The statistical literature on consumer credit scoring is long and covers many classification methods [1]; scorecards built on logistic regression are a standard practical tool [2]. In peer-to-peer (P2P) lending, individual investors rather than a bank bear the credit risk, so the platform has to publish a risk signal: Lending Club assigns every loan a grade (A to G) and a sub-grade (A1 to G5, 35 levels) and ties the interest rate to it [3].

The public Lending Club file used here (Kaggle dataset `wordsforthewise/lending-club` [4]) contains accepted loans issued between 2007 and 2018 with 151 columns: loan terms, borrower declarations, credit-bureau variables at the time of application, the repayment history after origination, hardship plans and debt-settlement fields. Two facts about the file shape the whole study. First, only some of the columns are knowable at application time (Section 3.4). Second, loans are observed until a snapshot date (December 2018 in this file), so recent loans have not had time to reach a final outcome (Section 3.2).

1.2 Challenges

1.2.0.1 Data leakage. A column is *leaky* if it is generated by, or informative only because of, the outcome that we want to predict. In this dataset, columns such as `recoveries`, `total_rec_prncp` or `last_fico_range_high` are recorded after origination. Section 4.3 measures how strong they are: in the training data the best post-origination column alone reaches a single-column ROC-AUC of 0.9074, versus 0.6565 for the best application-time column. A model that used them would look excellent and be useless for deciding on a new application.

1.2.0.2 Temporal dependence and dataset shift. The relationship between borrower characteristics and default is not constant over calendar time: macro-economic conditions, platform policy and the mix of borrowers change. Mixing periods in a random split lets the model see the future when predicting the past, which inflates offline scores [5].

In our own data the default rate drifts from 13.70% (train) to 14.74% (validation) and 14.82% (test), so even within three years the base rate moves.

1.2.0.3 Right-censoring and label maturity. A loan issued in 2017 with a 60-month term cannot have a final outcome in a December 2018 snapshot. Treating "not yet defaulted" as "repaid" would bias the labels. Section 3.2 shows that, by issue year, the share of loans with a final label falls from 100% (2011–2013) to 11.4% (2018).

1.2.0.4 An incumbent model. The platform's `sub_grade` is itself a risk model built by the lender. It is a highly informative application-time variable and a natural benchmark; including it also changes how other coefficients should be read (Section 4.7).

1.2.0.5 Messy and partly missing predictors. Credit-bureau fields appear at different dates, 26 retained columns are 5–30% missing in training, many are highly skewed (Section 4.5), and some pairs are almost duplicates (loan amount and instalment, $r = 0.994$).

1.2.0.6 Moderate class imbalance. After filtering, 13.7% of training loans are defaults; we therefore evaluate ranking quality (AUC, Gini, KS), which does not depend on a threshold or the base rate.

1.3 Motivation, objectives and approach

Beyond its practical interest, the dataset is a good teaching case: it forces decisions on what may be used (leakage), on whom the model is tested (out-of-time evaluation), against what (the incumbent) and how sure one can be of an improvement (confidence intervals). The objective is to estimate, honestly, how much application-time information adds to the platform's own sub-grade for ranking default risk. The approach, detailed in Chapter 3, is:

1. restrict to labelled, matured, 36-month loans issued from 2012 and split by issue date into train/validation/test (70/15/15);
2. whitelist application-time columns using a documented dictionary, and verify the whitelist against single-column AUC;
3. clean, engineer and encode features with a scikit-learn pipeline fitted only on the training set;
4. compare a `sub_grade` baseline with logistic regression, a decision tree, a random forest and histogram gradient boosting, tuned with time-ordered cross-validation inside the training set;

5. quantify differences with paired-bootstrap intervals on the validation set and evaluate the final models once on the test set;

6. study the role of `sub_grade` (removal and coefficient shift) and the cost of a compact 12-field scorecard.

1.4 Chapter summary

On Lending Club data the main pitfalls are leaky post-origination columns, temporal drift, label censoring and the platform's own grade as incumbent. The next chapter formalises the task and reviews related work.

CHAPTER 2. PROBLEM FORMULATION AND THEORETICAL BACKGROUND

This chapter states the prediction task formally, reviews related work on credit scoring with Lending Club data, and summarises the models, metrics and statistical tools that are used in the experiments.

2.1 Problem formulation

2.1.0.1 Input. For each loan i the input is a vector $\mathbf{x}_i$ of features known when the application was assessed: loan amount, borrower declarations (income, employment length, home ownership, purpose, state), credit-bureau variables (FICO range, debt-to-income ratio, delinquencies, inquiries, revolving utilisation, account counts, ...) and the platform's own `sub_grade`. Features that are only observed after the loan is made are excluded by construction (Section 3.4).

2.1.0.2 Output. A score $p_i = \hat{P}(y_i = 1 \mid \mathbf{x}_i) \in [0,1]$, where $y_i = 1$ if the loan ends as *Charged Off* and $y_i = 0$ if it ends as *Fully Paid*. The score is used to *rank* applicants; no decision threshold is fixed, because the threshold depends on the lender's costs and risk appetite.

2.1.0.3 Learning problem. Let t_i be the issue month of loan i and let $t_1 < t_2$ be two cut dates. Models are trained on $\{i : t_i < t_1\}$, compared on $\{i : t_1 \le t_i < t_2\}$ and finally evaluated once on $\{i : t_i \ge t_2\}$. The goal is to maximise the out-of-time ranking quality

$$\mathrm{AUC}(s) = \Pr\big(s(\mathbf{x}^+) > s(\mathbf{x}^-)\big) + \tfrac{1}{2}\Pr\big(s(\mathbf{x}^+) = s(\mathbf{x}^-)\big), \tag{2.1}$$

where $\mathbf{x}^+$ and $\mathbf{x}^-$ are the features of a randomly drawn defaulted and a randomly drawn fully-paid loan from the evaluation period [6, 7].

2.1.0.4 Reference model. The incumbent is the `sub_grade` baseline: the score of a loan is the training-set default rate of its sub-grade. It uses one application-time variable and no fitted parameters other than 35 group means.

2.2 Related work

2.2.1 Statistical credit scoring

Credit scoring has been studied for decades with discriminant analysis, logistic regression, trees and, more recently, ensembles. Hand and Henley's review of statistical methods for consumer credit scoring [1] surveys the classification methods used in the field, and the practitioner text by Siddiqi on scorecard development [2] covers building

scorecards, including the conversion of model output to a points scale with a "points to double the odds" parameter, which Section 2.4 uses.

2.2.2 Studies on Lending Club data

Serrano-Cinca et al. [3] analyse 24,449 Lending Club loans using univariate tests, survival analysis and a logistic regression. According to the authors, the factors explaining default include loan purpose, annual income, housing situation, credit history and indebtedness, and the platform-assigned grade is the single most predictive factor, with accuracy improving when other information—especially the borrower's debt level—is added. This is the literature precedent for treating the grade as the benchmark that other variables must improve upon.

Malekipirbazari and Aksakalli [8] propose random forests for borrower status on Lending Club data from January 2012 to September 2014, restricted by the authors to about 68 K loans that were fully paid or defaulted. They report that random forests outperform logistic regression, *k*-nearest neighbours and a support-vector classifier in their comparison (AUC about 0.71 versus 0.68 for logistic regression, as reported by the authors) and outperform FICO scores and Lending Club grades in identifying good borrowers. Their set-up differs from ours, so the figures are not comparable with Chapter 4.

Qian et al. [5] study *dataset shift* in credit scoring on Lending Club data (1,160,066 records, 2018–2020 in their file). They argue that mixing periods when training and validating leaks future information and inflates offline results, and propose adversarial validation to select training data resembling the data to be predicted. We adopt their motivation (strict chronological evaluation) but not their method.

2.2.3 Open-source projects

Several public repositories analyse the same file and are relevant to our design. We read their README pages and describe only their design, without quoting their scores, because their data, filters and splits differ from ours and we did not reproduce their code.

- `Arturo-GA/lendingclub-default-risk` [9]: out-of-time 70/15/15 split of loans sorted by origination date, restricted to loans whose term had expired by the end of the extract; logistic regression, a weight-of-evidence scorecard, random forest and gradient-boosting models. Our split and maturity rule follow the same idea.

- `vaibhavkev/credit-risk` [10]: 36-month loans issued 2012–2015, out-of-time train/validation/test, compared with Lending Club's interest rate and sub-grade.

- `Tanish-Srivastava/credit-risk-scorecard` [11]: weight-of-evidence logistic scorecard with a random split, using Lending Club's grade only as an external compari-

son.

- `Nasha14/Lending-Club-Loan-Default-Prediction` [12]: logistic regression versus XGBoost, with an experiment that removes `grade` and `sub_grade`; the README does not state the split protocol.

- `Arnav618/lending-club-credit-risk` [13]: tuned XGBoost on a stratified sample with a bootstrap interval for the AUC; no time-based split is described.

Of these, only the first two use a time-based evaluation, which is the main difference from the other repositories and the reason our protocol follows them.

2.2.4 Positioning and gap

The cited studies show that (i) the platform grade is a strong predictor and the right benchmark [3], (ii) non-linear ensembles can improve on logistic regression in some set-ups [8], and (iii) time-respecting evaluation matters [5]. The gap we address is a student-scale, fully reproducible study that combines all three: an out-of-time protocol, an explicit `sub_grade` baseline with bootstrap uncertainty, and an ablation that shows what happens to the other coefficients when the grade is removed.

2.3 Models

All models take the standardised and one-hot-encoded feature matrix of Section 3.8 as input and output a probability of default.

2.3.1 Logistic regression with L2 regularisation

Logistic regression models the log-odds of default as linear in the features, $\Pr(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ with $\sigma(z) = 1/(1+e^{-z})$. In scikit-learn the parameters minimise

$$\tfrac{1}{2}\|\mathbf{w}\|_2^2 + C \sum_{i=1}^{n} \Big[-y_i \log p_i - (1-y_i)\log(1-p_i) \Big], \tag{2.2}$$

so a *smaller C* means a *stronger* penalty. The squared-norm penalty is the ridge penalty of Hoerl and Kennard [14]; it shrinks coefficients, stabilises estimates when features are correlated and is the reason features must be standardised so the penalty treats them on a common scale. The intercept is not penalised. Coefficients are interpretable as changes in log-odds per one standard deviation of a feature (for numeric features) or per category (for one-hot columns).

2.3.2 Decision tree

A classification tree (CART [15]) recursively splits the feature space with axis-parallel thresholds chosen to reduce impurity, and predicts the default rate of the train-

ing loans in each leaf. It captures non-linearities but has high variance unless depth is limited; here `max_depth` is tuned and `min_samples_leaf` $= 50$.

2.3.3 Random forest

A random forest [16] averages many trees, each grown on a bootstrap sample of the training loans and, at each split, on a random subset of features. Averaging reduces variance relative to a single deep tree. The tuned hyperparameters are the number of trees, maximum depth and minimum leaf size.

2.3.4 Histogram gradient boosting

Gradient boosting [17] builds trees sequentially, each fitted to the gradient of the log-loss at the current predictions, and adds them with a shrinkage factor (`learning_rate`). The histogram variant bins each feature into at most 256 bins before searching splits, which makes training on hundreds of thousands of rows fast; scikit-learn's implementation follows the design of LightGBM [18, 19]. The tuned hyperparameters are the number of boosting iterations, maximum depth and learning rate.

2.4 Evaluation metrics and statistical tools

2.4.0.1 ROC-AUC and Gini. The ROC curve plots the true-positive rate against the false-positive rate as the score threshold varies; its area is the AUC of Equation (2.1). A random ranking has AUC 0.5. The Gini coefficient used in credit scoring is a linear rescaling,

$$\mathrm{Gini} = 2\,\mathrm{AUC} - 1. \tag{2.3}$$

2.4.0.2 KS statistic. The Kolmogorov–Smirnov statistic [20] is the maximum vertical gap between the cumulative score distributions of defaulters and non-defaulters; equivalently

$$\mathrm{KS} = \max_{\tau} \left[\mathrm{TPR}(\tau) - \mathrm{FPR}(\tau)\right]. \tag{2.4}$$

2.4.0.3 Overfitting check. For each model we report $\Delta_{\text{train-val}} = \mathrm{AUC}_{\text{train}} - \mathrm{AUC}_{\text{val}}$. A large positive gap indicates that the model fits the training period better than later loans. A negative gap is possible when the validation period is easier to rank than the training period; it is reported, not interpreted away.

2.4.0.4 Time-ordered cross-validation. Hyperparameters are chosen by `TimeSeriesSplit` with three splits: the training set (sorted by issue date) is divided into four consecutive blocks; fold k trains on the first k blocks and validates on block $k+1$ (the expanding-window scheme implemented by scikit-learn [19]). Unlike shuffled K-fold, no fold ever trains on loans issued after the loans it is scored on; time-respecting validation of depen-

dent data is discussed in [21], which proposes blocked cross-validation.

2.4.0.5 Paired bootstrap for AUC differences. To judge whether model A beats model B on a validation set of size n we resample the n loans with replacement $B = 1{,}000$ times [22]. For resample b we compute $\Delta^{(b)} = \mathrm{AUC}_A^{(b)} - \mathrm{AUC}_B^{(b)}$ on the *same* resampled loans (a paired design, which cancels the shared sampling noise). The 2.5% and 97.5% percentiles of $\{\Delta^{(b)}\}$ form a 95% interval; if it excludes 0 we call the difference statistically distinguishable from sampling noise. The interval reflects sampling variability of the validation loans only; it does not capture variability from refitting the models or from time.

2.4.0.6 Points-based scorecard. A model probability p is converted to points by

$$\mathrm{Score} = \mathrm{Offset} - \mathrm{Factor} \cdot \ln \frac{p}{1-p}, \qquad \mathrm{Factor} = \frac{\mathrm{PDO}}{\ln 2}, \quad \mathrm{Offset} = S_0 - \mathrm{Factor} \cdot \ln o_0, \tag{2.5}$$

where S_0 is the reference score at default odds o_0 and each additional PDO points halves the default odds. The conversion is the standard scorecard scaling described in [2]; the reference values ($\mathrm{PDO} = 20$, $S_0 = 600$ at $o_0 = 1/5$, i.e. a default rate of about 16.7%, scores clipped to $[300, 850]$) are our own choices and are not taken from that source or checked against any official standard.

2.5 Chapter summary

The task is a time-respecting binary ranking problem benchmarked against `sub_grade`; the models, metrics, time-ordered tuning and paired bootstrap defined here are applied in Chapters 3 and 4.

CHAPTER 3. DATA AND METHODOLOGY

This chapter describes the dataset, the definition of the label and the population, the out-of-time split, the leakage controls, the preprocessing pipeline, the models and tuning protocol, and the evaluation protocol. Each rule stated here is checked by a code cell whose printed output is reported in Chapter 4.

3.1 Dataset and data dictionary

The raw file is `accepted_2007_to_2018Q4.csv.gz` from the Kaggle dataset `wordsforthewise/` [4]. It contains 2,260,701 rows and 151 columns (3.29 GB in memory after loading). Because the columns are numerous and cryptically named, notebook 1 builds a data dictionary in which every column has (i) a one-line meaning, (ii) a group and (iii) a *hypothesis* about whether it is known at application time: `Yes`, `No` or `Uncertain`. Code cells assert that the dictionary covers every raw column exactly once (no missing, extra or duplicate names). The eight groups are: loan information (21 columns), borrower information (8), credit history part 1 (38) and part 2 (31), joint application (16), post-loan payments (15), hardship plan (15) and debt settlement (7), 151 in total.

The `Yes`/`No`/`Uncertain` labels are *hypotheses from column descriptions*, not facts; Section 3.4 lists how they are verified with data.

3.2 Label and population

3.2.0.1 Label. Of the nine `loan_status` values, only two are final outcomes that we use: `Fully Paid` $\rightarrow y = 0$ and `Charged Off` $\rightarrow y = 1$. To avoid relying only on the names of the statuses we check which statuses have no outstanding principal (`out_prncp`$= 0$ for all loans of that status): `Charged Off`, `Fully Paid` and the two "Does not meet the credit policy" statuses are finished; `Current`, `In Grace Period`, `Late` and `Default` are not. The two "credit policy" statuses (2,749 loans, 0.12% of the file) belong to an older lending policy and are excluded to keep the product homogeneous. 33 rows have no status (and no issue date) and are also dropped.

3.2.0.2 Population filters. Four filters are applied in sequence, each justified by code:

1. **Labelled**: status is `Fully Paid` or `Charged Off`.
2. **Issued from 2012** (`START_YEAR`=2012): loans before 2012 are only 2.96% of the labelled loans, and 44 numeric columns have no data before 2012 (Section 4.2).
3. **Matured**: with $m(\cdot)$ = year$\times 12$ + month, a loan is matured if $m(\text{issue}) + \text{term} \leq$

m(snapshot), where the snapshot is the latest issue month in the file (2018-12). A matured loan has had the whole contractual term to reach a final outcome. This rule is verified: 99.6% of matured loans have a final label versus 42.4% of the others.

4. **36-month term**: with the matured rule, 60-month loans can only appear for issue months up to 2013-12, so the 2015 validation and test periods would contain none; keeping them would change the term mix between train and test. Restricting to 36-month loans makes the term constant across all periods.

3.3 Out-of-time split

The filtered loans are sorted by (`issue_date`, `id`). Two cut *dates* are the issue dates found at positions $\lfloor 0.70n \rfloor$ and $\lfloor 0.85n \rfloor$ of the sorted data; train is every loan issued before the first date, validation every loan between the dates, test every loan on or after the second date. Cutting at whole dates (not at row positions) guarantees that no issue month is shared between sets, so the realised proportions are not exactly 70/15/15. Code asserts that $\max t_{\text{train}} < \min t_{\text{val}}$, $\max t_{\text{val}} < \min t_{\text{test}}$ and that no row is lost or duplicated. A `split_manifest.json` stores the cut dates, row counts and an MD5 hash of the sorted training ids so that other groups can check that they use the same split.

3.4 Leakage controls

Four rules protect against leakage.

1. **Application-time whitelist.** Only columns whose dictionary hypothesis is `Yes` may become features. Columns marked `No` (e.g. payments, recoveries, last FICO pull, hardship, settlement) and `Uncertain` (e.g. `funded_amnt`, `initial_list_status`) are excluded. For `funded_amnt`, the data show it equals the requested `loan_amnt` in 100% of training loans, so excluding it loses nothing.

2. **Empirical check.** For every suspect column and every whitelisted column we compute a single-column ROC-AUC on the training set. For numeric columns missing values are filled with the median; for text columns with at most 100 distinct values each value is replaced by its in-sample default rate (a diagnostic only, never a feature); other text columns are reduced to a "has a value" indicator; the reported AUC is $\max(\text{AUC}, 1-\text{AUC})$. If post-origination columns are far stronger than any application-time column, this is evidence that they carry outcome information.

3. **Split before fitting.** Everything that learns a number from data—medians, means, standard deviations, one-hot category lists and the caps `REVOL_UTIL_CAP`, `LOAN_TO_INCOME_CAP` thresholds such as `MAX_MISSING`—is computed on the training set only.

4. **EDA on training only.** All exploratory statistics that relate features to the label use

the training set; the single exception is the time-structure analysis of notebook 1, which uses all labelled loans but asks only about counts, maturity and term mix, not about feature–label relations.

3.5 Cleaning rules

Cleaning rules are fixed row-wise formulas, so they cannot leak information between sets. They are chosen from the meaning of the columns, and the thresholds that need data are computed on the training set.

- `annual_inc` ≤ 0 or `loan_amnt` $\leq 0 \rightarrow$ missing.
- `dti` < 0 or $> 100 \rightarrow$ missing (`DTI_MAX`=100; it is a percentage).
- `revol_util` is capped above at its 99.9th training percentile, 102.4 (values above 100 are possible because a card can exceed its limit, so they are not removed).
- `fico_range_low` > `fico_range_high` $\rightarrow$ both missing.
- `earliest_cr_line` later than the issue date $\rightarrow$ missing (to avoid negative credit history).
- duplicate `id` rows are dropped (none are found).

3.6 Column grouping and feature selection

All 151 columns are assigned to exactly one rule group, in this priority order: (1) label/auxiliary; (2) identifier (at least 99% distinct values and under 50% missing); (3) completely empty in train; (4) post-origination or uncertain timing; (5) joint-application; (6) text with more than 100 distinct values (not dates); (7) (quasi-)constant, i.e. one value in at least 99% of rows; (8) more than 30% missing in train (`MAX_MISSING`=0.30); (9) otherwise a *candidate*. The numeric candidates are then pruned for collinearity: for each pair with $|r| \geq 0.90$ in training data, the column with the lower single-column AUC is dropped. Finally, two choices are made on substantive grounds and justified in Chapter 4: `grade` is dropped because `sub_grade` contains it, and `int_rate` is dropped because Lending Club sets it by grade, so it duplicates the incumbent.

3.7 Feature engineering

Six features are created by fixed formulas (functions `make_*` in notebook 2):

$$\begin{aligned}
\text{emp_length_years} &= \text{years in } \texttt{emp_length} \text{ (“<1 year” } \to 0,\ \text{“10+ years” } \to 10),\\
\text{fico_avg} &= \tfrac{1}{2}(\text{fico_range_low} + \text{fico_range_high}),\\
\text{credit_history_years} &= (\text{issue_date} - \text{earliest_cr_line})/365.25,\\
\text{loan_to_income} &= \min(\text{loan_amnt}/\text{annual_inc},\ 0.5),\\
\text{log_annual_inc} &= \ln(1 + \text{annual_inc}), \qquad \text{log_revol_bal} = \ln(1 + \text{revol_bal}).
\end{aligned}$$

The cap 0.5 is the 99.9th percentile of the training ratio. The raw columns `fico_range_low/high`, `annual_inc`, `revol_bal`, `emp_length` and `earliest_cr_line` are replaced by their derived versions.

3.8 Preprocessing pipeline

The whole preprocessing is one scikit-learn `Pipeline`:

1. `FunctionTransformer(fix_invalid_values)` (cleaning rules);
2. `FunctionTransformer(add_features)` (feature engineering);
3. `ColumnTransformer`: numeric columns → `SimpleImputer(strategy="median")` → `StandardScaler`; categorical columns → `OneHotEncoder(handle_unknown="ignore")`.

The pipeline is `fit` on the training set only; validation and test are only `transform`ed. The input has 58 raw columns; the output has 153 columns (45 numeric + 108 dummies = 35 `sub_grade` + 6 `home_ownership` + 3 `verification_status` + 14 `purpose` + 50 `addr_state`). Median imputation and standardisation are chosen after checking the data (Section 4.5): most columns with missing values are right-skewed, so the mean is pulled away from a typical borrower, and the raw scales differ by more than five orders of magnitude, which matters for the L2 penalty of Equation (2.2).

3.9 Models and tuning protocol

Five models share the same 153-column input (the baseline uses only `sub_grade`). All hyperparameters are selected by `GridSearchCV` with `TimeSeriesSplit(n_splits=3)` and ROC-AUC scoring *inside the training set*; the validation set is not used to tune. Table 3.1 lists the grids. Random seed is 42 everywhere.

Table 3.1 Models and hyperparameter grids searched with `TimeSeriesSplit(3)` inside the training set.

Model	Grid (fixed settings in italics)	Combinations
Baseline `sub_grade`	none; default rate per sub-grade in train	0
Logistic regression (L2)	$C \in \{0.001, 0.01, 0.1, 1, 10\}$; *lbfgs, max_iter=1000*	5
Decision tree	`max_depth`$\in \{3, \ldots, 10\}$; *min_samples_leaf=50*	8
Random forest	`n_estimators`$\in \{50, 100, 150\}$, `max_depth`$\in \{6, 8, 10\}$, `min_samples_leaf`$\in \{30, 50, 60\}$	27
HistGradientBoosting	`max_iter`$\in \{50, 100, 150\}$, `max_depth`$\in \{4, 6, 8, 10\}$, `learning_rate`$\in \{0.05, 0.1, 0.15, 0.2\}$; *min_samples_leaf=50*	48

3.10 Evaluation protocol

For each model we compute ROC-AUC on train and validation, Gini and KS on validation, and the gap $\text{AUC}_{\text{train}} - \text{AUC}_{\text{val}}$. Paired-bootstrap intervals ($B = 1{,}000$ resamples of the validation loans, seed 42) are computed for each model's AUC minus the baseline's, and for HistGB minus logistic regression. The test set is evaluated one time at the end for the baseline, logistic regression and HistGB; the test set is not used for tuning or model selection. Conclusions are written by code from the computed variables (f-strings) in the notebook, so that wording cannot drift from the numbers.

3.11 Follow-up experiments

Both experiments are in separate notebooks under `experiences/` and reuse the matrices of notebook 2. They do not change the main results.

3.11.0.1 Experiment 1: removing `sub_grade`. Remove the 35 `sub_grade` dummies (118 columns remain), refit logistic regression with $C = 1$ (not re-tuned), compare with the full model and the baseline on validation and test, bootstrap the AUC differences on validation, and tabulate how the coefficients of eight key numeric features change.

3.11.0.2 Experiment 2: compact scorecard. Use 10 numeric fields (`fico_avg`, `dti`, `log_annual_inc`, `loan_to_income`, `revol_util`, `inq_last_6mths`, `acc_open_past_24mths`, `total_bal_ex_mort`, `delinq_2yrs`, `loan_amnt`) and 2 categorical fields (`home_ownership`, `purpose`), i.e. 30 columns after encoding, refit logistic regression with $C = 1$, convert

probabilities to points with Equation (2.5) (PDO = 20, 600 points at default odds 1/5) and group scores into four bands whose cut-offs (590, 615, 640) were set by looking at the validation score distribution; the bands are then applied unchanged to the test set. The test set is used here for descriptive reporting, not for choosing among models; this is stated in the notebook.

3.12 Implementation and reproducibility

All code lives in three main notebooks (`01_data_split`, `02_features`, `03_models`) and two experiment notebooks; there is no separate `.py` module, and functions are defined near their first use. Libraries are pandas, NumPy, scikit-learn [19], Matplotlib, pyarrow and joblib (Python $\geq$ 3.10). Notebooks are committed with their outputs and must be run from top to bottom. The raw file is git-ignored and required; no synthetic data are used. Intermediate artefacts (split parquet files, the fitted pipeline, feature lists, the split manifest) are written to `data/processed` and `artifacts`, both git-ignored.

3.13 Chapter summary

The method restricts the data to labelled, matured, 36-month loans from 2012, splits them chronologically, admits only application-time columns (verified by single-column AUC), fits all learned preprocessing on the training set inside one pipeline, tunes five models with time-ordered cross-validation, and evaluates with AUC, Gini, KS, paired-bootstrap intervals and one final test evaluation. Chapter 4 reports what the code printed.

CHAPTER 4. EXPERIMENTS AND RESULTS

This chapter reports what the notebooks computed. Unless stated otherwise, every number is copied from a notebook output, and every figure is the image saved by the notebook. We first describe the setup and the data (Sections 4.1–4.3), then the evidence behind the preprocessing choices (Sections 4.4 and 4.5), the model comparison (Section 4.6), the two follow-up experiments (Section 4.7), a discussion, and the limitations (Section 4.9).

4.1 Experimental setup

The experiments use the protocol of Chapter 3: out-of-time split by issue date, 153 features after the pipeline of Section 3.8, hyperparameters tuned with `TimeSeriesSplit(3)` inside the training set, metrics ROC-AUC, Gini and KS, 1,000 paired-bootstrap resamples on validation, and a single test evaluation. The seed is 42. Metrics are reported for ranking quality only; no threshold-dependent metric (accuracy, F1) is used because no decision threshold is defined for the task.

4.2 Dataset statistics and the time structure

4.2.0.1 Labels. Of 2,260,701 loans, 1,345,310 have a final label: 1,076,751 Fully Paid (80.04%) and 268,559 Charged Off (19.96%) among the labelled loans. Table 4.1 lists all statuses. The statuses with no outstanding principal (`out_prncp`$= 0$ for every loan) are exactly the finished ones: `Fully Paid` (1,076,751 loans), `Charged Off` (268,559) and the two old-policy statuses (1,988 and 761); the unfinished ones (`Current`, `Late`, `In Grace Period`, `Default`) have outstanding principal in at least 98.7% of loans. This supports using `Fully Paid` and `Charged Off` as final outcomes.

Table 4.1 Loan statuses in the raw file and the share of loans with outstanding principal (`out_prncp` > 0).

loan_status	Loans	% of file	Share with principal
Fully Paid	1,076,751	47.63	0.000
Current	878,317	38.85	0.995
Charged Off	268,559	11.88	0.000
Late (31–120 days)	21,467	0.95	0.999
In Grace Period	8,436	0.37	0.987
Late (16–30 days)	4,349	0.19	0.992
Does not meet the credit policy: Fully Paid	1,988	0.09	0.000
Does not meet the credit policy: Charged Off	761	0.03	0.000
Default	40	0.00	1.000
(missing)	33	0.00	—

4.2.0.2 Time structure. Table 4.2 gives, by issue year, the number of loans, the fraction with a final label, the default rate among labelled loans and the share of 60-month loans. The labelled fraction is 100% for 2011–2013 and falls to 89.2% for 2015, 67.5% for 2016, 38.2% for 2017 and 11.4% for 2018, which is the right-censoring problem of Section 3.2: recent loans cannot be used without a maturity rule. The default rate among labelled loans also rises from 0.162 in 2012 to 0.233 in 2016, but the 2016–2018 values mix loans whose outcomes are largely unknown, so they should not be read as trends.

Table 4.2 Loans per issue year: share with a final label, default rate among labelled loans and share of 60-month loans.

Year	Loans	Labelled share	Default rate (labelled)	60-month share
2007	603	0.416	0.179	0.000
2008	2,393	0.653	0.158	0.000
2009	5,281	0.893	0.126	0.000
2010	12,537	0.920	0.129	0.270
2011	21,721	1.000	0.152	0.351
2012	53,367	1.000	0.162	0.185
2013	134,814	1.000	0.156	0.255
2014	235,629	0.947	0.184	0.310
2015	421,095	0.892	0.202	0.328
2016	434,407	0.675	0.233	0.255
2017	443,579	0.382	0.231	0.278
2018	495,242	0.114	0.157	0.304

4.2.0.3 Why start in 2012. Loans before 2012 are 39,786 labelled loans, 2.96% of all labelled loans. Among the numeric columns, 40 first have data in 2007, 44 in 2012, 16 in 2015 and 12 in 2017 (one column, `member_id`, is always empty). Figure 4.1 shows the loan volume per year and the year in which numeric columns appear; starting in 2012 loses under 3% of the labelled loans while the 44 columns that start in 2012 have data.

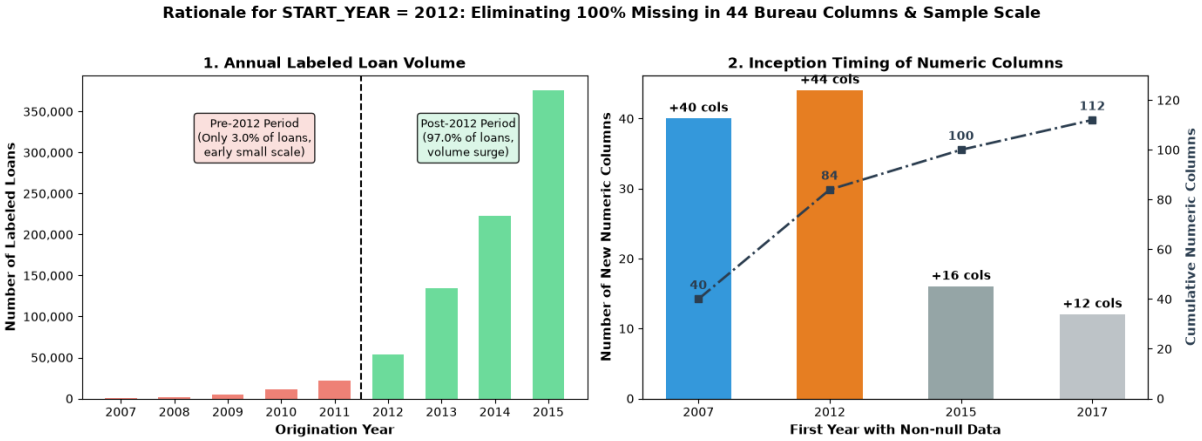


Figure 4.1 Left: number of labelled loans per issue year up to 2015, with the pre-2012 share. Right: number of numeric columns that first have data in each year, with the cumulative count.

4.2.0.4 Maturity rule and filters. Table 4.3 verifies the matured-loan rule: 99.6% of the 676,459 matured loans have a final label against 42.4% of the 1,584,242 unmatured ones. The default rate in the matured group is 14.8% (against 25.1% for the unmatured loans that happen to be labelled), and only 8.2% of matured loans are 60-month, which is why Section 3.2 restricts to 36-month loans. Table 4.4 shows the funnel: 589,488 loans, 26.1% of the file, remain; the 36-month filter removes 7.0% of the loans left after the maturity rule.

Table 4.3 Check of the matured-loan rule.

Group	Loans	Labelled share	Default rate	60-month share
Unmatured	1,584,242	0.424	0.251	0.376
Matured	676,459	0.996	0.148	0.082

Table 4.4 Filtering funnel.

Step	Rows	Retained (%)	Step reduction (%)
Raw data	2,260,701	100.00	0.00
Labelled (Fully Paid / Charged Off)	1,345,310	59.51	40.49
and issued from 2012	1,305,524	57.75	2.96
and matured	633,767	28.03	51.46
and 36-month term	589,488	26.08	6.99

4.2.0.5 The split. Table 4.5 gives the three sets. The cut dates are 2015-06-01 and 2015-10-01. The training set holds 69.2% of the loans, spans 41 months and has a 13.7% default rate; validation and test are 15.7% and 15.0% with default rates 14.7% and 14.8%. All three sets contain only 36-month loans, so the term mix is identical. The manifest hash of the sorted training ids is `8ce37fef31257778922ffe920c32511c`. Figure 4.3 shows loan counts and default rate by issue month with the cut lines, and Figure 4.2 shows the label balance in the training set (86.3% repaid, 13.7% defaulted) and the default rate of each set. The default-rate gap between the largest and smallest set is 0.011, i.e. a mild but real drift that the models will face.

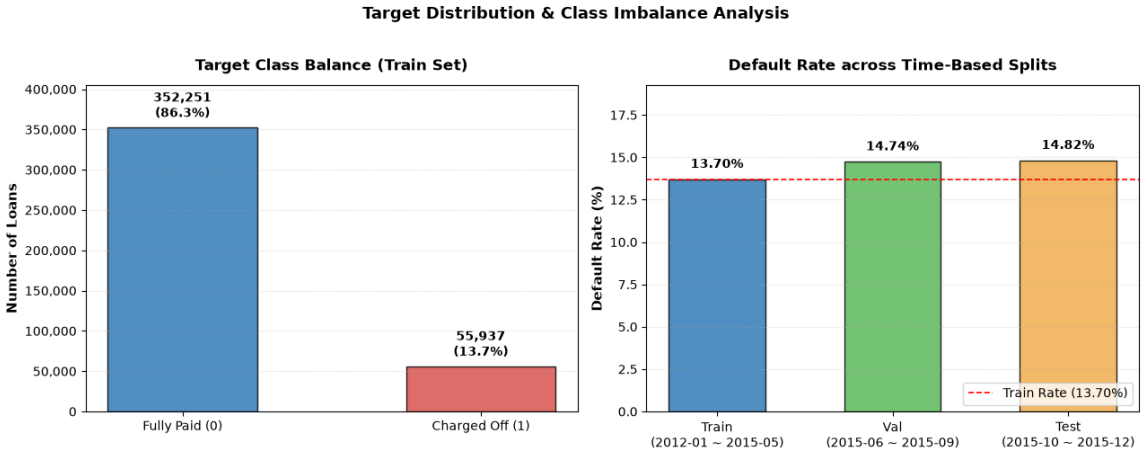


Figure 4.2 Left: label balance in the training set. Right: default rate in each time-based set.

Table 4.5 Train, validation and test sets.

Set	Loans	% of total	From	To	Default rate
Train	408,188	69.24	2012-01	2015-05	0.137
Validation	92,631	15.71	2015-06	2015-09	0.147
Test	88,669	15.04	2015-10	2015-12	0.148

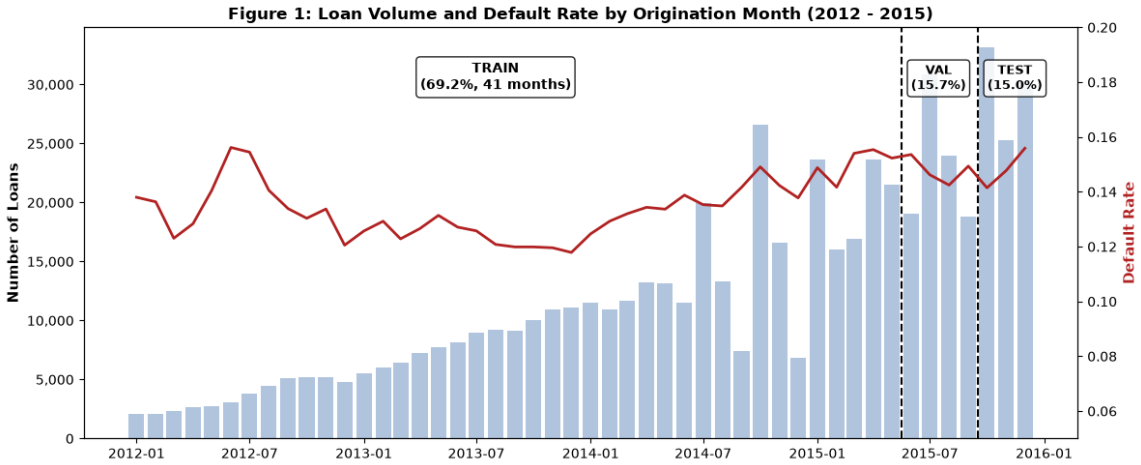


Figure 4.3 Number of loans (bars) and default rate (line) by issue month, 2012–2015, with the two cut dates separating train, validation and test.

4.2.0.6 Label maturity at the end of the window. For 36-month loans issued in 2015 the final-label rate is 100.0% in every month from January to October, 99.9% in November and 99.7% in December (99.68%). The test period therefore has about 0.3% of its December loans still without a final outcome; this is a known, small limitation (Section 4.9).

4.2.0.7 Missing values. In the training set, 53 columns have no missing values, 66 are partially missing and 32 are entirely empty (Figure 4.4). Seventeen of the 32 empty columns do have values in validation or test because Lending Club began recording them after the training period; a model cannot learn from them, so they are excluded. The `desc` column shows the largest train–test shift in missing share (18.4 percentage points).

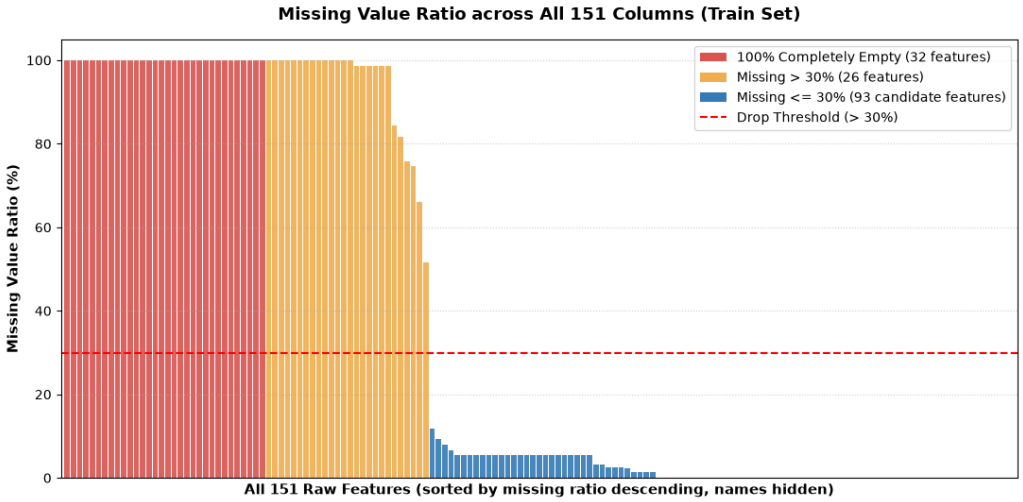


Figure 4.4 Share of missing values for each of the 151 raw columns in the training set (sorted; names hidden). The dashed line marks the 30% drop threshold.

4.3 Evidence for the leakage controls and the column groups

Table 4.6 lists the ten suspect (post-origination or uncertain-timing) columns with the highest single-column training AUC. Of the 42 suspect columns, 10 exceed the best application-time column, which is `sub_grade` with AUC 0.6565 (73 application-time columns were scored). The top suspects reach 0.9074 (`last_fico_range_low` and `last_fico_range_high`), 0.9054 (`recoveries`) and 0.8677 (`collection_recovery_fee`). These columns record what happened *after* the loan, so a model that used them would appear far better than any application-time model; the check supports the whitelist rule empirically. Note that the AUC of text columns with few values is computed in-sample (Section 3.4), which may overstate the single-column strength of `sub_grade` (it is scored on the same data from which its group means are taken), so 0.6565 should be read with that caveat.

Table 4.6 Suspect columns with the highest single-column ROC-AUC on the training set (best application-time column: `sub_grade`, 0.6565).

Column	AUC	Meaning
last_fico_range_low	0.9074	Lower FICO bound at the latest credit pull
last_fico_range_high	0.9074	Upper FICO bound at the latest credit pull
recoveries	0.9054	Amount recovered after charge-off
collection_recovery_fee	0.8677	Collection fee after charge-off
last_credit_pull_d	0.8355	Month of the latest credit pull
total_rec_prncp	0.8280	Principal received to date
last_pymnt_amnt	0.7673	Amount of the latest payment
total_pymnt	0.7554	Total payments received to date
total_pymnt_inv	0.7554	Total payments received (investor portion)
last_pymnt_d	0.6577	Month of the latest payment

Table 4.7 and Figure 4.5 show the outcome of the rule groups: from 151 columns, 57 candidates (38%) remain. The 42 post-origination or uncertain columns and the 32 columns empty in training are the largest removed groups. All 16 joint-application columns are empty in the training set (every training loan is an `Individual` application; the test set contains 0.3% `Joint App`), so they fall under the "empty" rule. A code assertion confirms that no column of the leakage group is among the 57 candidates.

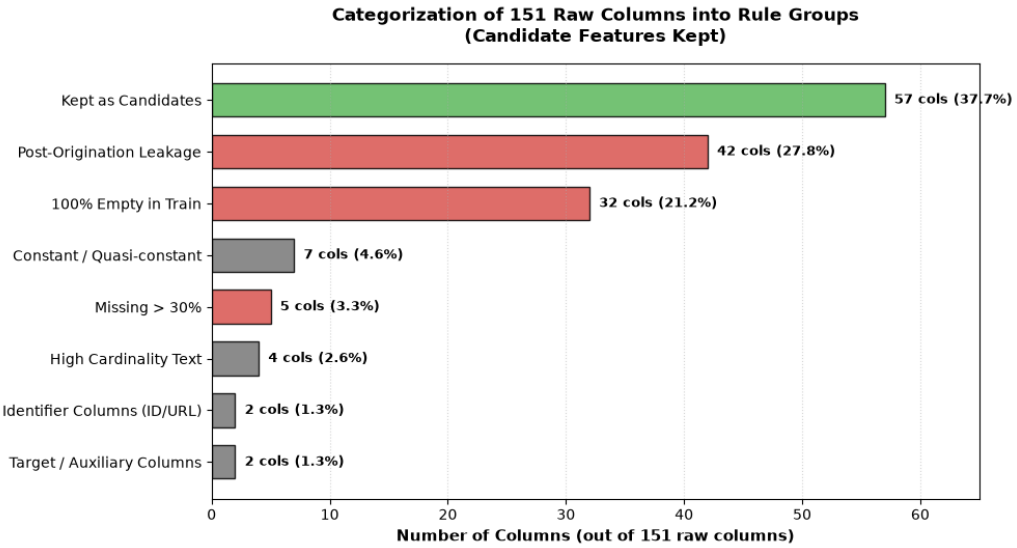


Figure 4.5 Categorisation of the 151 raw columns into rule groups; green marks the 57 candidate columns.

Table 4.7 Number of the 151 raw columns in each rule group.

Group	Columns
Kept as candidates	57
Post-origination / uncertain timing	42
Empty in train	32
(Quasi-)constant	7
Missing $>30\%$ in train	5
High-cardinality text	4
Identifier	2
Label / auxiliary	2
Total	**151**

Because the five columns in the "missing $>30\%$" group are the `mths_since_...` variables (months since last delinquency, public record, major derogatory item, bankcard delinquency and revolving delinquency), their information is lost; it is a deliberate simplification, and an indicator-based treatment is listed as future work. Of the remaining candidates, 26 columns are 5–30% missing in the training set; all of them first appear in the data in 2012 and are median-imputed.

4.4 Cleaning checks, EDA and feature selection

4.4.0.1 Logic checks. Table 4.8 counts violations of the business rules before and after cleaning. The rules `annual_inc` ≤ 0, `dti` <0 and `dti` >100 have no violations in training but do occur in validation/test (2 and 5 rows in test); the rules were set from the meaning of the columns, not from validation or test. After cleaning, the corrected rules are at zero in all three sets; `revol_util` >100 (1,321 / 421 / 461 rows) is kept because it is possible, only the extreme values are capped at 102.4. There are no duplicate or cross-set loan ids.

Table 4.8 Rows violating each logic rule before cleaning (after cleaning, only `revol_util`>100 remains because it is a valid case).

Rule	Train	Validation	Test
annual_inc ≤ 0	0	0	2
dti < 0	0	0	0
dti > 100	0	0	5
revol_util > 100	1,321	421	461
fico_range_low $>$ fico_range_high	0	0	0
first credit line after issue date	0	0	0
loan_amnt ≤ 0	0	0	0

4.4.0.2 Default rate versus key variables. Figure 4.6 shows the default rate by loan purpose in the training set: it is highest for `small_business` (22.4%) and lowest for `car` (11.4%) and `credit_card` (11.5%); the `educational` bar is 0.0% (the notebook does not print the number of such loans, so we do not interpret it). Figure 4.7 shows that the default rate rises monotonically with sub-grade, from 2.7% for A1 to 36.4% for G5: the platform's grade is a strong, ordered risk signal. Figure 4.8 shows that Lending Club's interest rate is tiered by grade (it rises with the grade), which is why `int_rate` would duplicate the incumbent score and is dropped.

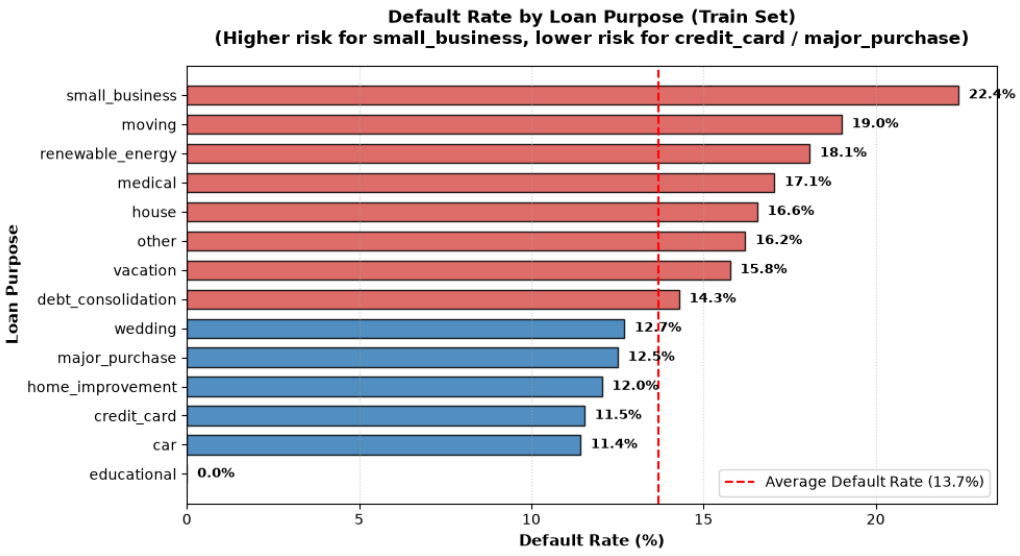


Figure 4.6 Default rate by loan purpose in the training set (dashed line: overall 13.7%).

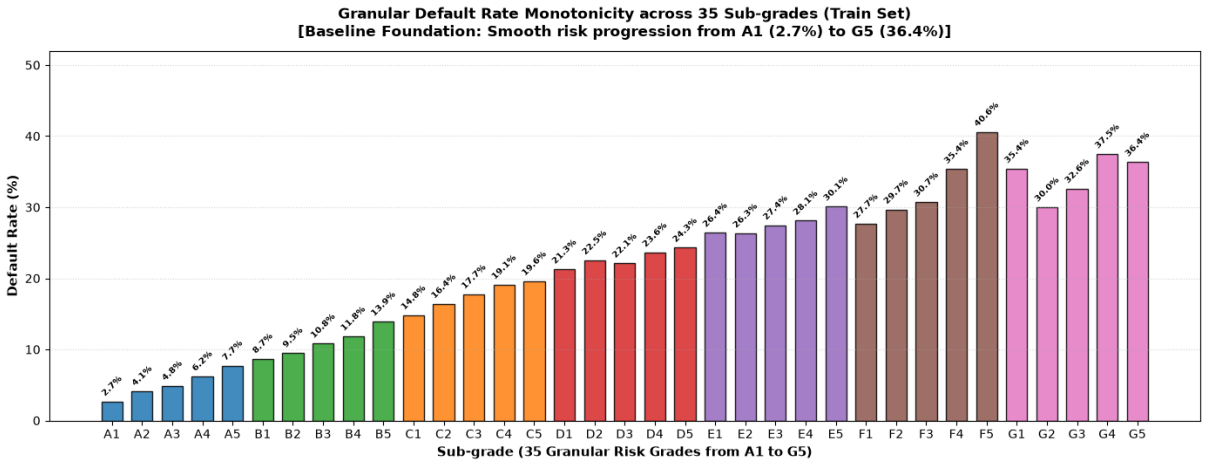


Figure 4.7 Default rate by sub-grade (A1–G5) in the training set.

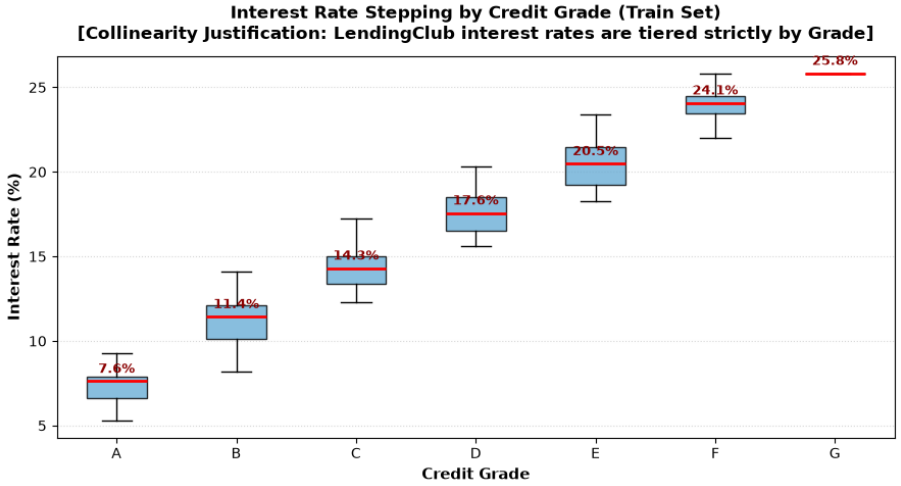


Figure 4.8 Distribution of the interest rate by grade in the training set; rates step up with the grade.

4.4.0.3 Behaviour of the main numeric features. Figure 4.9 gives the default rate within each decile of six features (training set, overall rate 13.7%). Risk falls steadily as `fico_avg` rises (18.6% in the lowest decile, median FICO 662, to 5.7% in the highest, median 762) and as `annual_inc` rises (20.0% to 9.1%), and rises with `dti` (10.5% to 19.1%), `revol_util` (10.3% to 16.2%) and `loan_to_income` (10.7% to 18.5%). `loan_amnt` is *not* monotone: its default rate peaks at 15.7% in decile 4 and ends at 12.4% in decile 10. A linear term in the logistic model therefore cannot capture the loan-amount pattern, which is one reason non-linear models are worth testing. Figure 4.10 compares the engineered `fico_avg` and `loan_to_income` between repaid and defaulted loans: the median FICO is lower and the median loan-to-income higher for defaulted loans.

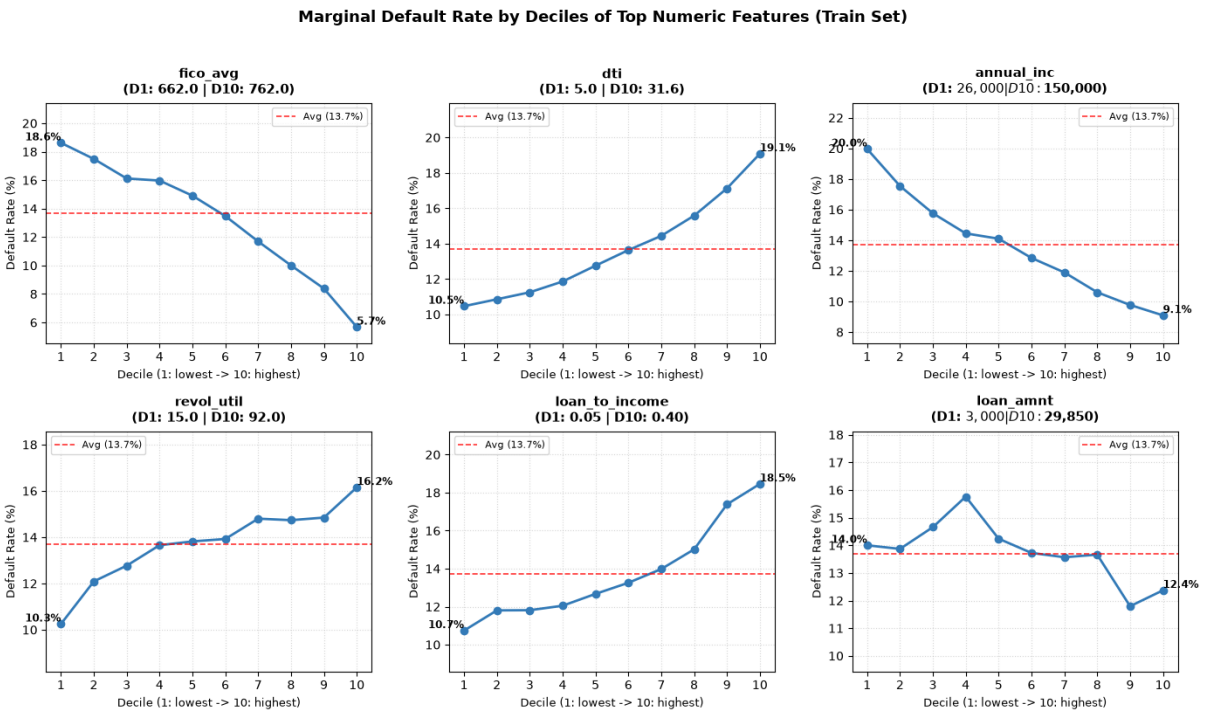


Figure 4.9 Default rate by decile of six numeric features in the training set (dashed line: overall default rate). Titles give the median value of the first and last decile; matplotlib renders the two dollar signs in some titles as mathematical text.

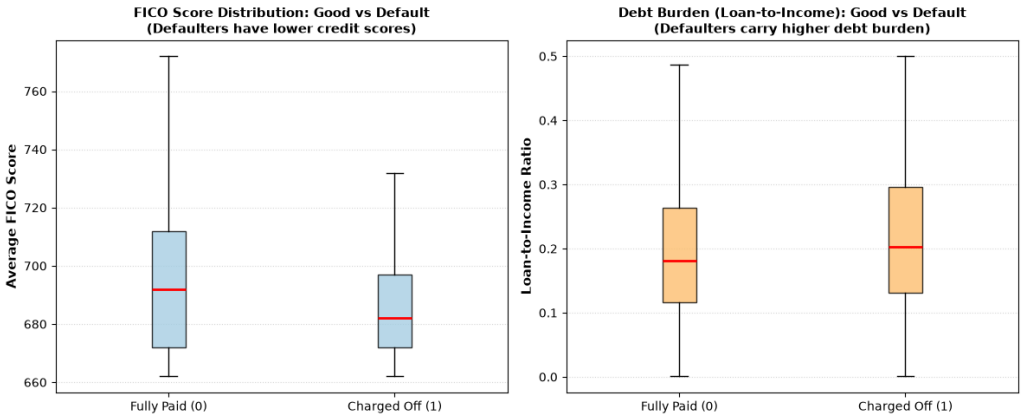


Figure 4.10 Box plots (outliers hidden) of `fico_avg` and `loan_to_income` for repaid and defaulted loans in the training set.

4.4.0.4 Single-feature strength and collinearity. Figure 4.11 ranks the numeric candidates by single-column AUC on the training set: `int_rate` (0.650) is the strongest, followed by `fico_avg` (0.598) and `log_annual_inc` (0.580); `int_rate` is shown only for comparison, as it was later dropped. The best purely numeric application-time feature thus has an AUC of about 0.60, compared with 0.6565 for `sub_grade` alone, which already indicates that a model must combine many weak signals to beat the incumbent.

Five numeric pairs have $|r| \geq 0.90$ in the training set (Table 4.9; Figure 4.12; `loan_amnt` and `installment` have $r = 0.994$); in each pair the column with the lower single-column AUC is dropped (`open_acc`, `installment`, `num_rev_tl_bal_gt_0`, `tot_cur_bal`, `credit_history_years`). Together with `int_rate` and the constant `term_months`, this reduces 52 numeric candidates to 45; with the five categorical columns (`sub_grade`, `home_ownership`, `verification_status`, `purpose`, `addr_state`) the model has 50 input features and 153 columns after one-hot encoding.

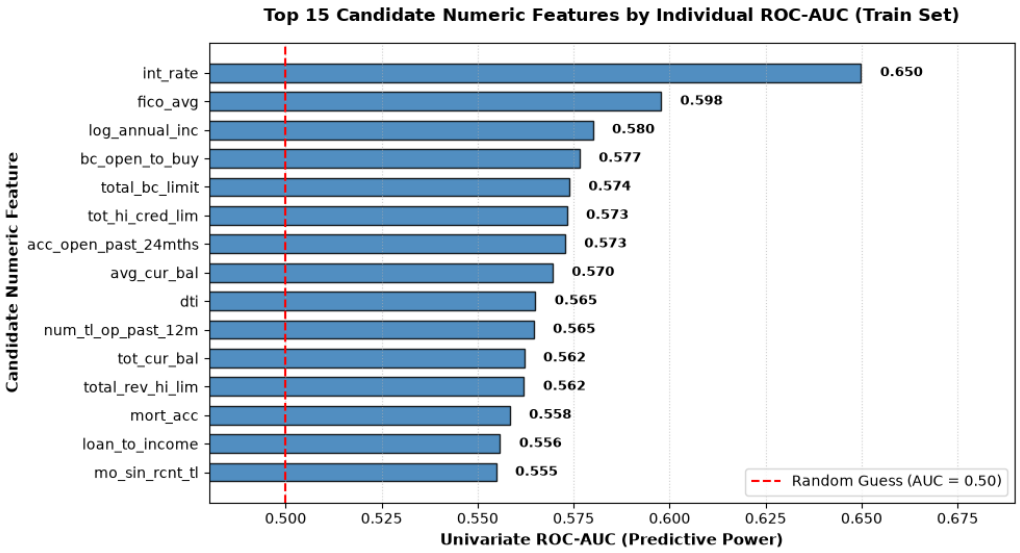


Figure 4.11 Top 15 numeric candidates by single-column ROC-AUC in the training set.

Table 4.9 Numeric column pairs with $|r| \geq 0.90$ in the training set and the column dropped.

Column A	Column B	r	AUC A	AUC B	Dropped
open_acc	num_sats	0.9987	0.5065	0.5071	open_acc
loan_amnt	installment	0.9941	0.5179	0.5044	installment
num_actv_rev_tl	num_rev_tl_bal_gt_0	0.9903	0.5398	0.5395	num_rev_tl_bal_gt_0
tot_cur_bal	tot_hi_cred_lim	0.9744	0.5621	0.5732	tot_cur_bal
mo_sin_old_rev_tl_op	credit_history_years	0.9245	0.5492	0.5400	credit_history_years

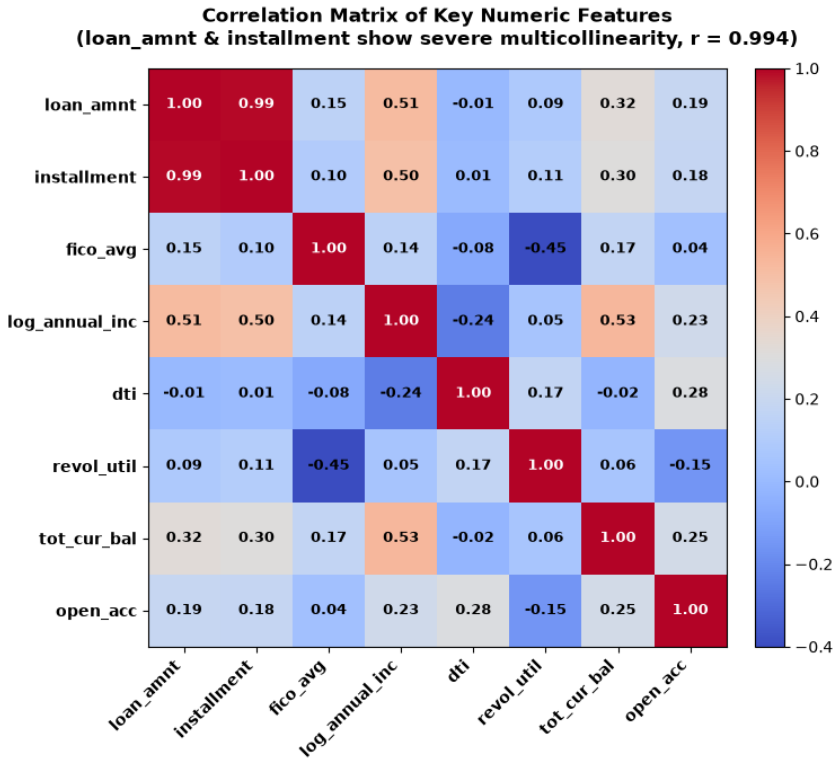


Figure 4.12 Pearson correlation matrix of eight key numeric features in the training set; `loan_amnt` **and** `installment` **have** $r = 0.994$**.**

4.5 Evidence for the preprocessing choices

4.5.0.1 Median versus mean imputation. Of the 45 numeric features, 31 have missing values in the training set. Of these, 28 are right-skewed (skewness > 0) and 25 strongly so (skewness > 1); the most extreme are `tot_coll_amt` (skewness 606.4) and `total_rev_hi_lim` (97.2). For several balance and limit columns the mean is far above the median (for example `avg_cur_bal`: mean 12,502.7 versus median 6,228.0, +100.7%; `tot_hi_cred_lim`: 156,218.6 versus 91,316.0, +71.1%). Figure 4.13 summarises the skewness and mean/median ratio for the most skewed columns, and Figure 4.14 shows that filling with the mean creates an artificial spike away from the bulk of the distribution (`total_rev_hi_lim`), whereas the median fill sits within it. We therefore use median imputation.

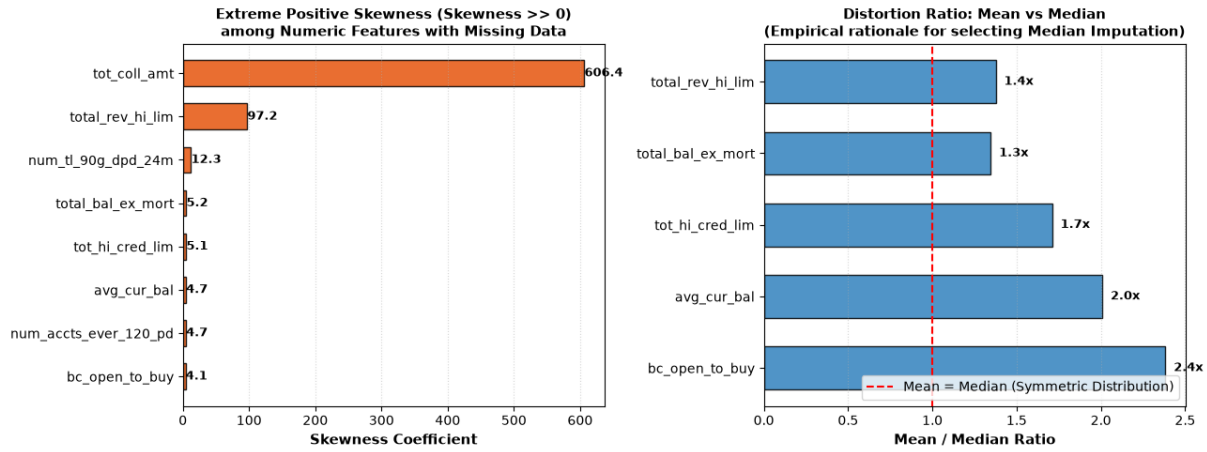


Figure 4.13 Left: skewness of the most skewed numeric features with missing data. Right: mean-to-median ratio for the same features.

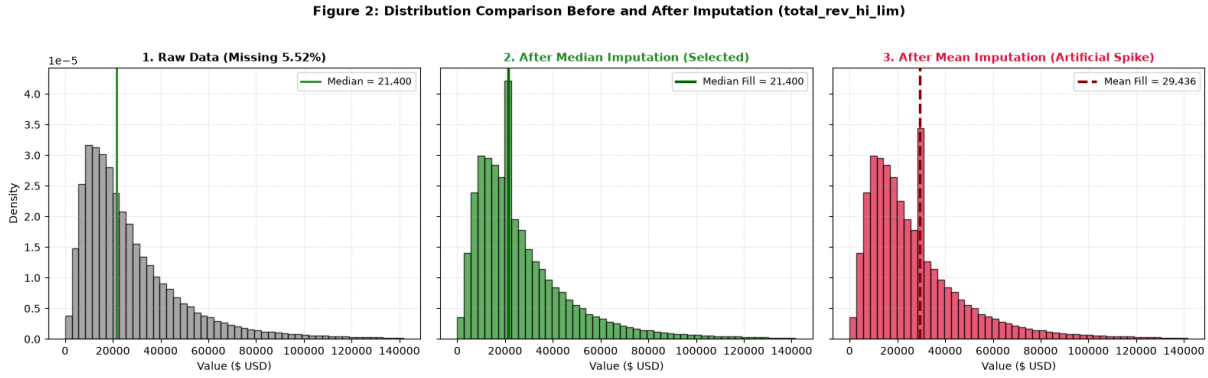


Figure 4.14 Distribution of `total_rev_hi_lim` (below the 99th percentile): raw data, after median imputation and after mean imputation.

4.5.0.2 Standardisation. On a 10,000-loan sample of the training set, the raw means range from 0.34 (`delinq_2yrs`) to 127,162 (`tot_cur_bal`), a ratio above 10^5 (Figure 4.15, logarithmic axis). After `StandardScaler` every feature has mean about 0 (maximum $|\mu| = 5.07 \times 10^{-16}$) and standard deviation 1.0001 (Figure 4.16). This matters for Equation (2.2), because the L2 penalty would otherwise penalise features unequally depending on their units, and it makes coefficient sizes comparable.

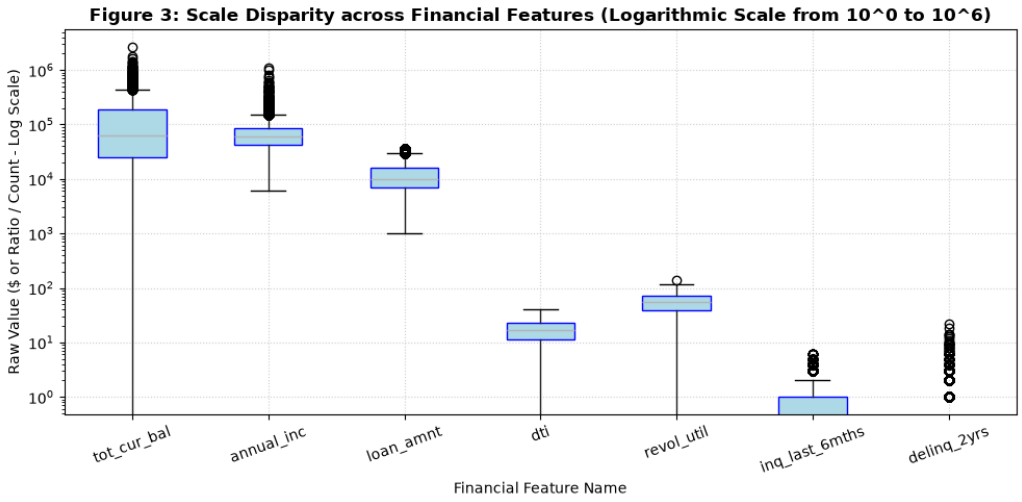


Figure 4.15 Raw scale of seven financial features (logarithmic axis) in a sample of 10,000 training loans.

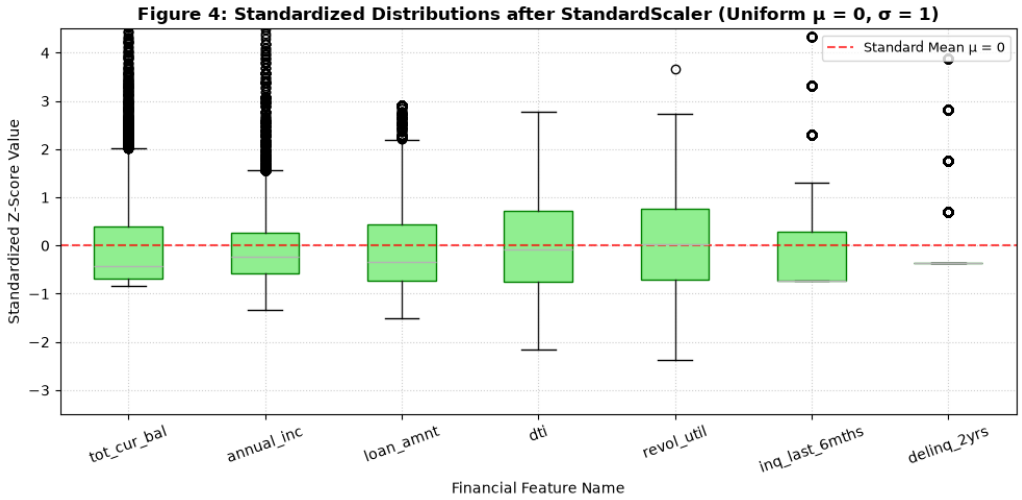


Figure 4.16 The same features after standardisation.

4.5.0.3 No leakage through the pipeline. The pipeline output has 153 columns for train, validation and test and no remaining missing values. After scaling, the training means are zero (maximum absolute mean 6.39×10^{-16}) whereas the validation means are not (maximum absolute mean 0.162), as expected when validation is transformed with training statistics. A second fit on the training set reproduces the same validation matrix (`allclose` is true), so the preprocessing is deterministic.

4.6 Model comparison

4.6.1 Hyperparameter tuning

Table 4.10 summarises the time-ordered cross-validated ROC-AUC (mean over three folds, inside the training set) for the selected configurations. For logistic regression

the CV AUC is 0.6855 at $C = 1$ and 0.6854 at $C = 10$, 0.6853 at $C = 0.1$, 0.6839 at $C = 0.01$ and 0.6782 at $C = 0.001$: the score is flat for $C \geq 0.1$, and the largest penalty is clearly worse. For the decision tree the best depth is 7 (CV AUC 0.6355), with shallower and deeper trees lower (depth 3: 0.6139; depth 10: 0.6278). The best random forest has 150 trees, depth 10 and minimum leaf 50 (0.6741); the best gradient-boosting model has 150 iterations, depth 10 and learning rate 0.05 (0.6852). In the random-forest grid the depth-10 settings occupy the top nine places (CV AUC 0.6720–0.6741), and the first depth-8 configuration is 0.6710; in the boosting grid, the ten best configurations lie within 0.0018 of each other (CV AUC 0.6852 to 0.6834).

Table 4.10 Selected hyperparameters and cross-validated ROC-AUC (`TimeSeriesSplit(3)` inside the training set).

Model	Selected hyperparameters	CV AUC
Logistic regression	$C = 1.0$	0.6855
Decision tree	`max_depth`=7, `min_samples_leaf`=50	0.6355
Random forest	150 trees, `max_depth`=10, `min_samples_leaf`=50	0.6741
HistGradientBoosting	`max_iter`=150, `max_depth`=10, `learning_rate`=0.05	0.6852

The random-forest and boosting selections sit on the edge of their grids (trees 150, depth 10; boosting iterations 150, depth 10, learning rate 0.05), so a wider search could find better settings; Section 4.9 discusses this.

4.6.2 Validation performance and overfitting check

Table 4.11 gives the validation results with the train–validation AUC gap, and Figure 4.17 plots the ROC curves and the three metrics. The `sub_grade` baseline reaches validation AUC 0.6815 (Gini 36.31%, KS 27.02%). Logistic regression (0.7040) and HistGradientBoosting (0.7054) are the best two models; their Gini values are 40.79% and 41.07% and their KS values 29.94% and 30.02%. The random forest (0.6876) is above the baseline but below both; the decision tree (0.6536) is below the baseline.

Table 4.11 Model comparison on the validation set (92,631 loans) and train–validation AUC gap.

Model	AUC train	AUC val	Gap (train–val)	Gini val	KS val
Baseline `sub_grade`	0.6565	0.6815	−0.0250	36.31%	27.02%
Logistic L2 ($C = 1$)	0.6873	0.7040	−0.0167	40.79%	29.94%
Decision tree (depth 7)	0.6541	0.6536	+0.0006	30.71%	22.25%
Random forest (150, depth 10)	0.7060	0.6876	+0.0184	37.52%	27.21%
HistGradientBoosting	0.7096	0.7054	+0.0042	41.07%	30.02%

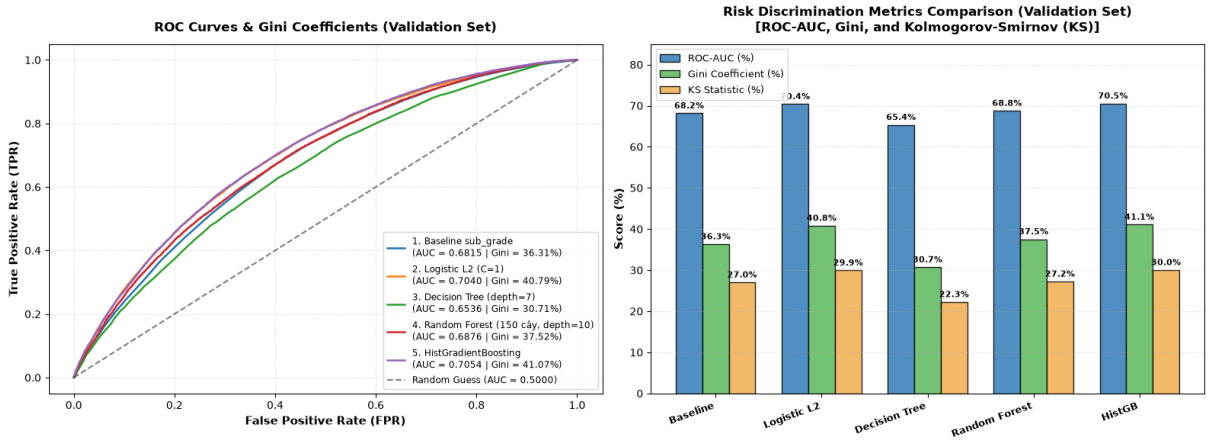


Figure 4.17 Validation set. Left: ROC curves with AUC and Gini of the five models. Right: ROC-AUC, Gini and KS of each model.

The overfitting check gives a mixed picture. The random forest has the largest positive gap (+0.0184): it fits the training period noticeably better than the later validation period. HistGradientBoosting has a gap of +0.0042 and the decision tree +0.0006. For the baseline (−0.0250) and logistic regression (−0.0167) the *validation* AUC is higher than the training AUC. We did not isolate the cause (a change over time in how sharply sub-grade separates defaults would be one untested candidate), so we only report the observation.

The KS values in Table 4.11 are the maximum vertical gap between the cumulative score curves of defaulters and non-defaulters, shown in Figure 4.18 for the baseline and logistic regression.

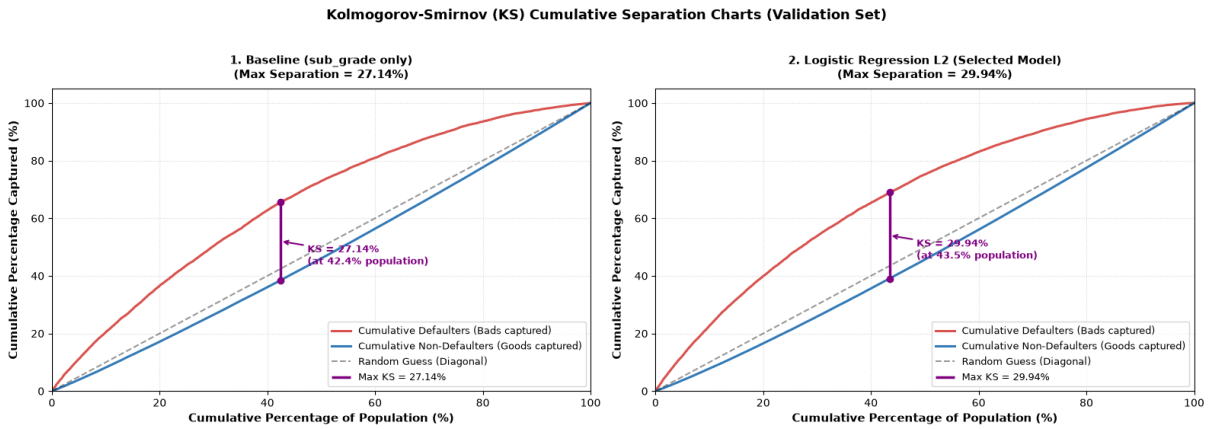


Figure 4.18 Cumulative separation (KS) charts on the validation set for the baseline (left) and logistic regression (right).

4.6.3 Bootstrap confidence intervals

Table 4.12 gives the paired-bootstrap results on the validation set (1,000 resamples). Logistic regression improves the AUC over the baseline by $+0.0225$ (95% CI $[+0.0196, +0.0253]$), HistGradientBoosting by $+0.0238$ ($[+0.0206, +0.0270]$) and the random forest by $+0.0061$ ($[+0.0024, +0.0102]$); all three intervals exclude zero. The decision tree is below the baseline (-0.0280, $[-0.0329, -0.0233]$). Comparing the two best models directly gives HistGB minus logistic regression $= [-0.0002, +0.0031]$, an interval that contains zero: on this validation set we cannot distinguish them, even though the point estimate favours boosting by about $+0.0014$.

Table 4.12 Paired-bootstrap difference in validation AUC against the `sub_grade` baseline ($B = 1{,}000$).

Model	Mean ΔAUC	95% CI
Logistic L2 ($C = 1$)	$+0.0225$	$[+0.0196, +0.0253]$
Decision tree (depth 7)	-0.0280	$[-0.0329, -0.0233]$
Random forest (150, depth 10)	$+0.0061$	$[+0.0024, +0.0102]$
HistGradientBoosting	$+0.0238$	$[+0.0206, +0.0270]$
HistGB − Logistic L2	—	$[-0.0002, +0.0031]$

4.6.4 Final evaluation on the test set

The test set (88,669 loans issued October–December 2015) was evaluated once for the baseline, logistic regression and HistGradientBoosting (Table 4.13). The baseline reaches AUC 0.6818 (Gini 36.36%, KS 26.45%); logistic regression 0.7039 (Gini 40.79%, KS 29.87%), i.e. $+0.0221$; HistGradientBoosting 0.7055 (Gini 41.10%, KS 29.97%), i.e. $+0.0237$. The test values are almost identical to the validation values

(baseline 0.6815 versus 0.6818; logistic regression 0.7040 versus 0.7039; HistGB 0.7054 versus 0.7055), so the ranking of the three models and the size of the gain are stable from validation to test. In Gini terms the models add about 4.4 (logistic) and 4.7 (HistGB) points over the baseline.

Table 4.13 Single evaluation on the test set (88,669 loans, 2015-10 to 2015-12).

Model	AUC	Gini	KS	ΔAUC vs. baseline
Baseline `sub_grade`	0.6818	36.36%	26.45%	—
Logistic L2 ($C = 1$)	0.7039	40.79%	29.87%	+0.0221
HistGradientBoosting	0.7055	41.10%	29.97%	+0.0237

The random forest and decision tree were not evaluated on the test set because they were not candidates after validation. The notebook states that logistic regression is preferred for interpretability and speed because its performance is statistically indistinguishable from boosting; that preference is a modelling judgement and not a result of the bootstrap.

4.6.5 Logistic coefficients

Because features are standardised, logistic coefficients are changes in log-odds per standard deviation (numeric) or per category. The largest positive coefficients are `sub_grade` dummies of grades E and F (`E5` 0.5212, `E4` 0.4084, `E3` 0.3878, `F1` 0.3704) and `purpose_small_business` (0.3115); the most negative are `sub_grade` A1 (-1.2025), A2 (-0.9408), A3 (-0.8183), then `home_ownership_MORTGAGE` (-0.4011), `addr_state_NH` (-0.3820) and `verification_status_Not Verified` (-0.3708). The extremes are dominated by `sub_grade`: when the platform's grade is in the model it takes the largest weights, which motivates Experiment 1.

4.7 Follow-up experiments

4.7.1 Experiment 1: removing `sub_grade`

Table 4.14 compares the baseline, the full logistic model (153 features) and the logistic model without the 35 `sub_grade` dummies (118 features). Without `sub_grade` the validation AUC falls from 0.7040 to 0.6890 and the test AUC from 0.7039 to 0.6879, but both stay above the baseline (0.6815 and 0.6818). The paired bootstrap gives, for no-sub-grade minus baseline, a mean $+0.0075$ with 95% CI $[+0.0033, +0.0117]$ (excluding zero), and for full minus no-sub-grade a mean $+0.0150$ with CI $[+0.0133, +0.0167]$. So the other application-time features alone beat the grade-based rule, by a smaller margin (on test $+0.0061$ against $+0.0221$ for the full model, i.e. roughly 28% of its gain), and adding `sub_grade` contributes a further $+0.015$ AUC. Figure 4.19 shows the ROC curves

and the metric bars.

Table 4.14 Effect of removing `sub_grade` (logistic regression, $C = 1$ for both variants).

Model	Feat.	AUC train	AUC val	AUC test	Gini test	KS test
Baseline `sub_grade`	1	0.6565	0.6815	0.6818	36.36%	26.45%
Logistic, with `sub_grade`	153	0.6873	0.7040	0.7039	40.79%	29.87%
Logistic, without `sub_grade`	118	0.6775	0.6890	0.6879	37.58%	27.34%

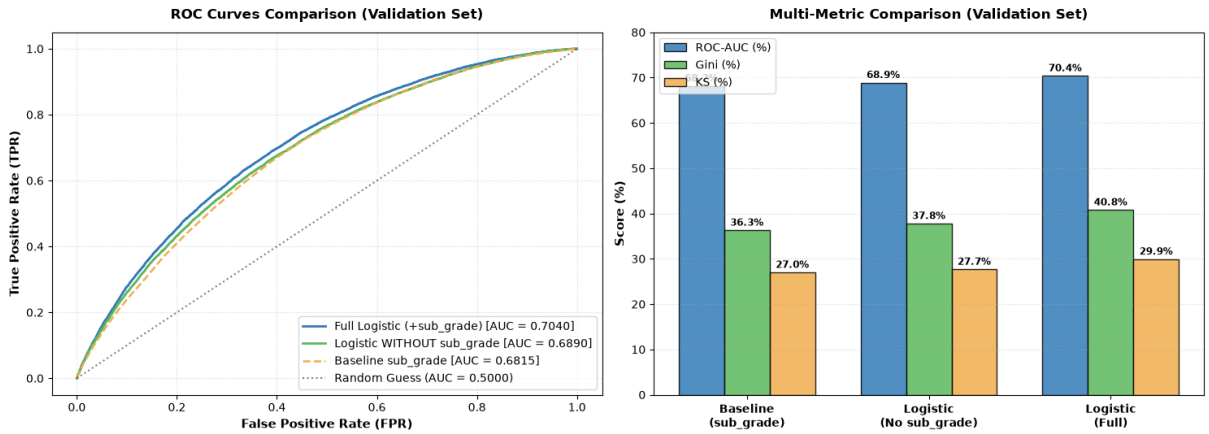


Figure 4.19 Validation ROC curves (left) and AUC/Gini/KS bars (right) for the baseline, logistic without `sub_grade` and logistic with all features.

Table 4.15 shows how the coefficients of eight key numeric features change when `sub_grade` is removed. Four coefficients grow by a factor of 1.9 to 2.6: `fico_avg` from -0.1067 to -0.2374 ($2.22\times$), `log_annual_inc` from -0.0386 to -0.1021 ($2.64\times$), `inq_last_6mths` from $+0.0494$ to $+0.0951$ ($1.92\times$) and `revol_util` from $+0.0521$ to $+0.0978$ ($1.88\times$). Others change little: `acc_open_past_24mths` $1.12\times$, `dti` $1.24\times$, `loan_to_income` $0.88\times$ and `mort_acc` $0.98\times$. This pattern is consistent with `sub_grade` *absorbing* part of the signal that these variables carry: when the grade is available, the model leans on it and gives smaller weights to the variables it partly encodes. It is an association in a fitted model, not a causal statement about how the platform builds the grade. Figure 4.20 shows the eight coefficients and the scatter of all 118 shared coefficients.

Table 4.15 Standardised logistic coefficients of eight numeric features with and without `sub_grade`.

Feature	With	Without	Shift	Ratio
fico_avg	−0.1067	−0.2374	−0.1307	2.22
dti	+0.1277	+0.1583	+0.0306	1.24
log_annual_inc	−0.0386	−0.1021	−0.0634	2.64
revol_util	+0.0521	+0.0978	+0.0457	1.88
inq_last_6mths	+0.0494	+0.0951	+0.0457	1.92
loan_to_income	+0.1491	+0.1310	−0.0181	0.88
acc_open_past_24mths	+0.1587	+0.1785	+0.0198	1.12
mort_acc	−0.0141	−0.0138	+0.0002	0.98

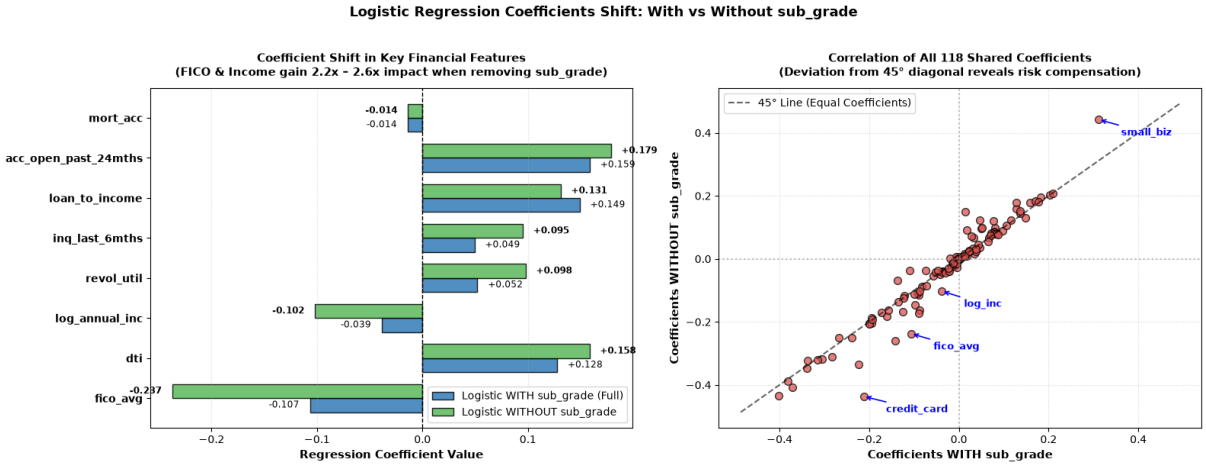


Figure 4.20 Left: coefficients of eight numeric features with and without `sub_grade`. Right: all 118 coefficients present in both models (dashed line: unchanged).

4.7.2 Experiment 2: compact 12-field scorecard

The compact model uses 12 fields (30 columns after encoding) and no `sub_grade`. Table 4.16 shows its performance: validation AUC 0.6750 (Gini 34.99%, KS 25.63%) and test AUC 0.6730 (Gini 34.60%, KS 25.07%). This is *below* the `sub_grade` baseline on both sets (−0.0066 on validation, −0.0088 on test). Relative to the full 153-feature model, the compact model keeps 84.8% of the test Gini (34.60% versus 40.79%) with 30 instead of 153 columns (about 80% fewer).

Table 4.16 Compact 12-field logistic model versus the full model and the baseline.

Model	AUC val	Gini val	KS val	AUC test	Gini test	KS test
Baseline `sub_grade`	0.6815	36.31%	27.02%	0.6818	36.36%	26.45%
Full logistic (153 columns)	0.7040	40.79%	29.94%	0.7039	40.79%	29.87%
Compact logistic (30 columns)	0.6750	34.99%	25.63%	0.6730	34.60%	25.07%

With PDO = 20 and 600 points at default odds 1/5, the validation scores range from 494 to 757 (mean 610.7, median 610; 10th, 50th and 93rd percentiles 588, 610 and 640). Using cut-offs 590, 615 and 640 (chosen from the validation distribution) the four bands behave as in Table 4.17. On validation the observed default rate falls from 29.38% in the lowest band to 3.45% in the highest; on the untouched test set the same cut-offs give 30.18% and 3.26%, with intermediate bands 17.28% and 8.66%. The ordering and the approximate levels hold out of time. The lowest band holds 11.4% of test loans and the highest 7.5%, against an overall test default rate of 14.82%. Figure 4.21 shows the score distributions of repaid and defaulted test loans and the ten numeric weights of the scorecard. The action labels attached to the bands in the notebook (automatic rejection, manual review, ...) are illustrative; no cost or profit analysis was performed, so we make no claim about their business value.

Table 4.17 Score bands of the compact scorecard: share of loans and observed default rate (cut-offs chosen on validation, applied unchanged to test).

	Validation			Test		
Band (points)	Loans	Share	Default	Loans	Share	Default
< 590	11,279	12.2%	29.38%	10,145	11.4%	30.18%
590–614	44,394	47.9%	17.08%	42,171	47.6%	17.28%
615–639	30,379	32.8%	8.32%	29,671	33.5%	8.66%
≥ 640	6,579	7.1%	3.45%	6,682	7.5%	3.26%

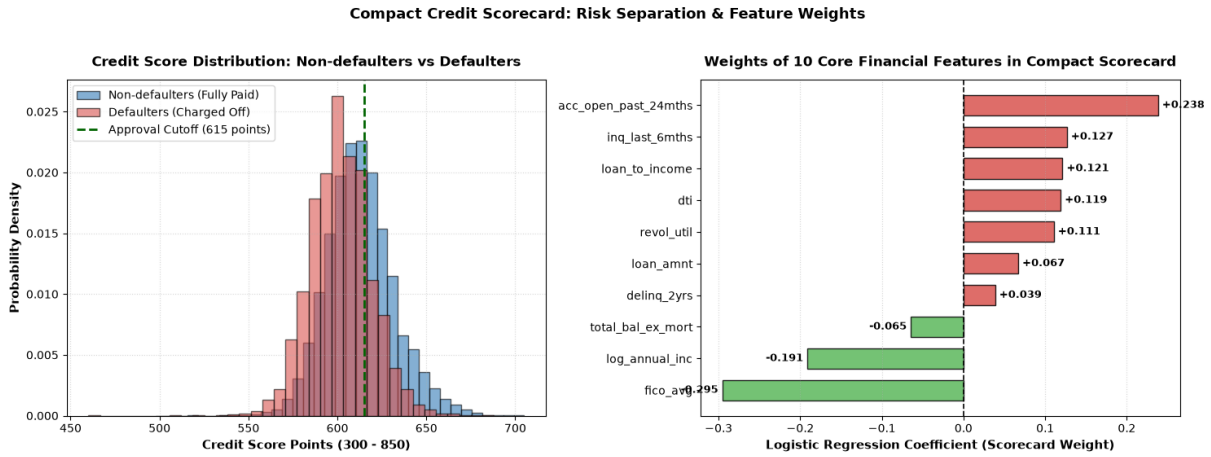


Figure 4.21 Left: density of the scorecard points for repaid and defaulted test loans. Right: weights of the ten numeric features in the compact scorecard.

4.8 Discussion

4.8.0.1 Does application-time information beat the incumbent? Yes, with the grade included: logistic regression and boosting improve test AUC by $+0.022$ to $+0.024$ (Gini by 4.4 to 4.7 points), and the validation intervals exclude zero. Without the grade the other features still beat the grade-based rule, but only by $+0.0061$ AUC on test ($+0.0075$ on validation, CI $[+0.0033, +0.0117]$). Both findings matter: the sub-grade is a strong reference, and what the extra variables add is modest in absolute terms. This ordering is in line with earlier Lending Club work reporting the grade as the most predictive single factor with accuracy improving when other information is added [3], but we make no numerical comparison with other studies, since their data, filters and splits differ.

4.8.0.2 Linear versus non-linear. Boosting was nominally best (validation AUC 0.7054 versus 0.7040), yet the paired bootstrap interval for the difference $[-0.0002, +0.0031]$ includes zero, and the test values (0.7055 versus 0.7039) differ by 0.0016. The data therefore do not show a benefit of non-linear structure beyond what the regularised linear model already captures, although the decile analysis showed non-monotone behaviour in `loan_amnt`. The random forest was clearly worse than both ($+0.0061$ against the baseline, with the largest overfitting gap), possibly because its tuned grid ended at the edge (depth 10) while the time-ordered CV favoured deeper trees; we did not test this. The single shallow tree was worse than the baseline. Choosing logistic regression is justified by parsimony and interpretability, not by higher accuracy.

4.8.0.3 Results that did not go the expected way. (i) The compact 12-field scorecard is below the baseline in AUC, so it is useful only where the platform's grade is unavailable, and even then at a cost of about 0.009 test AUC relative to the grade rule and 0.031 rela-

tive to the full model. (ii) The validation AUC of the baseline and of logistic regression is higher than their training AUC, which we cannot explain with the evidence at hand. (iii) The test set confirms the validation conclusions but cannot extend them, because it covers only three months.

4.9 Limitations and threats to validity

1. **Population.** The results apply to accepted, matured, 36-month Lending Club loans issued 2012–2015. Rejected applicants are not in the file, so we cannot say how the models behave on them, and 60-month loans are excluded.

2. **Short test window.** The test set covers three months (October–December 2015), and about 0.3% of December 2015 loans have no final outcome by the snapshot. Conclusions about the stability over longer periods or in other economic conditions are not supported.

3. **Temporal drift.** The default rate rises from 13.7% to 14.8% across the sets; we did not test or model dataset shift (for example with adversarial validation [5]).

4. **Tuning grids at their edges.** The selected random-forest and boosting settings lie on the grid boundary, and neighbouring configurations differ by about 0.001 in CV AUC. Better settings may exist, which could change the random-forest result and possibly the boosting–logistic comparison.

5. **Uncertainty quantification.** The bootstrap resamples the validation loans only: it ignores the variability from refitting, from the choice of hyperparameters and from time, it was not repeated on the test set, and a single random seed is used.

6. **Test-set use.** The test set is used once for model comparison in notebook 3 and is not used for tuning. The two follow-up notebooks also report test metrics for descriptive purposes after the main selection; they do not change which models are chosen, but the "single use" claim applies to the main comparison only.

7. **Missing data.** Median imputation is applied to 26 columns that are 5–30% missing in training, without missing-indicator variables, and five `mths_since_...` columns were removed for being more than 30% missing; we did not study whether missingness itself is informative.

8. **Probabilities and decisions.** Only ranking metrics are evaluated. Calibration, thresholds, profit and cost trade-offs were not examined; the PDO scaling constants are conventions we did not validate against any standard.

9. **Role of `sub_grade`.** `sub_grade` is an application-time variable produced by the plat-

form's own model. Including it makes the problem "improve on the incumbent"; excluding it (Experiment 1) makes it "replace the incumbent". Both are reported, but the decision of which to use in practice depends on the use case.

10. **Scope.** We did not analyse disparate impact (the models use `addr_state` and `home_ownership`) and used no external dataset for validation.

4.10 Chapter summary

On 589,488 matured 36-month loans split by time, logistic regression and histogram gradient boosting improve the out-of-time AUC over the sub-grade baseline by about 0.022–0.024 (test AUC 0.7039 and 0.7055 versus 0.6818), with bootstrap intervals that exclude zero, and they are statistically indistinguishable from each other; the random forest improves less, and a single tree does worse than the baseline. Without `sub_grade`, logistic regression still beats the baseline but only by 0.0061 on test, and the weights of FICO, income, inquiries and utilisation roughly double. A compact 12-field scorecard falls below the baseline yet separates risk bands from 3.26% to 30.18% observed default. The limitations include a short test window, grid-edge tuning, validation-only bootstrap intervals and an unexplained validation-above-training AUC for two models.

CONCLUSION AND FUTURE WORK

Summary of the work

This project asked whether application-time information improves, out of time, on the platform's own sub-grade for ranking Lending Club loans by default risk. We defined the label and a matured, 36-month, 2012–2015 population of 589,488 loans; split them chronologically into 408,188 / 92,631 / 88,669 loans; whitelisted application-time columns and confirmed with single-column AUC that post-origination columns are far stronger (up to 0.9074 against 0.6565); built a pipeline whose learned statistics come from the training set only; and compared a sub-grade baseline with L2 logistic regression, a decision tree, a random forest and histogram gradient boosting, tuned by time-ordered cross-validation.

Main findings

1. **The incumbent is beaten, modestly.** On the single test evaluation the baseline has AUC 0.6818, logistic regression 0.7039 and gradient boosting 0.7055 (Gini 36.36%, 40.79%, 41.10%). Bootstrap intervals on validation exclude zero for both models against the baseline.

2. **More complex is not clearly better.** Boosting and logistic regression cannot be distinguished (validation ΔAUC interval $[-0.0002, +0.0031]$); the random forest gains less and overfits more; a depth-7 tree is worse than the baseline.

3. **The grade absorbs signal.** Without `sub_grade`, logistic regression still beats the baseline (test AUC 0.6879, $+0.0061$) but the coefficients of FICO score, income, inquiries and utilisation roughly double (1.9 to 2.6 times), and adding `sub_grade` contributes $+0.0150$ validation AUC.

4. **A compact scorecard is cheap but weaker.** Twelve fields (30 columns) give test AUC 0.6730, below the baseline, but still produce risk bands whose observed default rates run from 30.18% to 3.26% on test.

Limitations

The conclusions hold for 36-month loans of 2012–2015 and a three-month test window. Tuning grids for the random forest and boosting end at their boundaries, the bootstrap intervals describe validation-sampling noise only, calibration and decision costs were not studied, and the higher-than-training validation AUC of the baseline and logistic regression is unexplained (Section 4.9).

Future work

- widen or randomise the hyperparameter search for the random forest and boosting, and repeat the logistic-versus-boosting comparison;
- extend the evaluation window (rolling-origin evaluation over several test periods) and test for dataset shift, for example with adversarial validation [5];
- treat the missing `mths_since_...` variables and the partially missing bureau fields with indicator variables or tree-native missing handling, and test whether missingness is informative;
- add calibration analysis (reliability curves, Brier score) and cost-sensitive thresholds so that scores can drive accept/reject decisions;
- include 60-month loans using a survival or time-to-default formulation rather than discarding unfinished loans;
- examine fairness and stability of the scorecard across regions and borrower groups.

REFERENCES

[1] D. J. Hand and W. E. Henley, "Statistical classification methods in consumer credit scoring: a review," *Journal of the Royal Statistical Society: Series A*, vol. 160, no. 3, pp. 523–541, 1997.

[2] N. Siddiqi, *Intelligent Credit Scoring: Building and Implementing Better Credit Risk Scorecards*, 2nd ed., ser. Wiley and SAS Business Series. Hoboken, NJ: John Wiley & Sons, 2017.

[3] C. Serrano-Cinca, B. Gutiérrez-Nieto, and L. López-Palacios, "Determinants of default in P2P lending," *PLoS ONE*, vol. 10, no. 10, p. e0139427, 2015.

[4] N. George, "Lending club loan data (accepted and rejected loans, 2007–2018)," Kaggle dataset (user `wordsforthewise`), https://www.kaggle.com/datasets/wordsforthewise/lending-club, accessed 2026.

[5] H. Qian, B. Wang, P. Ma, L. Peng, S. Gao, and Y. Song, "Managing dataset shift by adversarial validation for credit scoring," arXiv preprint arXiv:2112.10078, 2021.

[6] J. A. Hanley and B. J. McNeil, "The meaning and use of the area under a receiver operating characteristic (ROC) curve," *Radiology*, vol. 143, no. 1, pp. 29–36, 1982.

[7] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[8] M. Malekipirbazari and V. Aksakalli, "Risk assessment in social lending via random forests," *Expert Systems with Applications*, vol. 42, pp. 4621–4631, 2015.

[9] Arturo-GA, "lendingclub-default-risk," GitHub repository, https://github.com/Arturo-GA/lendingclub-default-risk, description based on the repository's README page; code and reported results were not reproduced.

[10] vaibhavkev, "credit-risk," GitHub repository, https://github.com/vaibhavkev/credit-risk, description based on the repository's README page; code and reported results were not reproduced.

[11] Tanish-Srivastava, "credit-risk-scorecard," GitHub repository, https://github.com/Tanish-Srivastava/credit-risk-scorecard, description based on the repository's README page; code and reported results were not reproduced.

[12] Nasha14, "Lending-club-loan-default-prediction," GitHub repository, https://github.com/Nasha14/Lending-Club-Loan-Default-Prediction, description based on the repository's README page; code and reported results were not reproduced.

[13] Arnav618, "lending-club-credit-risk," GitHub repository, https://github.com/Arnav618/lending-club-credit-risk, description based on the repository's README page; code and reported results were not reproduced.

[14] A. E. Hoerl and R. W. Kennard, "Ridge regression: biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[15] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[16] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[17] J. H. Friedman, "Greedy function approximation: a gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[18] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: a highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 3146–3154.

[19] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[20] F. J. Massey, "The Kolmogorov–Smirnov test for goodness of fit," *Journal of the American Statistical Association*, vol. 46, no. 253, pp. 68–78, 1951.

[21] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.

[22] B. Efron, "Bootstrap methods: another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

APPENDIX

A. Final model inputs

Table 4.18 lists the 45 numeric features and 5 categorical features that enter the pipeline (153 columns after one-hot encoding).

Table 4.18 Input features of the final pipeline.

Type	Features
Numeric, raw (40)	loan_amnt, dti, delinq_2yrs, inq_last_6mths, pub_rec, revol_util, total_acc, collections_12_mths_ex_med, tot_coll_amt, total_rev_hi_lim, acc_open_past_24mths, avg_cur_bal, bc_open_to_buy, bc_util, mo_sin_old_il_acct, mo_sin_old_rev_tl_op, mo_sin_rcnt_rev_tl_op, mo_sin_rcnt_tl, mort_acc, mths_since_recent_bc, mths_since_recent_inq, num_accts_ever_120_pd, num_actv_bc_tl, num_actv_rev_tl, num_bc_sats, num_bc_tl, num_il_tl, num_op_rev_tl, num_rev_accts, num_sats, num_tl_90g_dpd_24m, num_tl_op_past_12m, pct_tl_nvr_dlq, percent_bc_gt_75, pub_rec_bankruptcies, tax_liens, tot_hi_cred_lim, total_bal_ex_mort, total_bc_limit, total_il_high_credit_limit
Numeric, engineered (5)	emp_length_years, fico_avg, loan_to_income, log_annual_inc, log_revol_bal
Categorical (5)	sub_grade (35 levels), home_ownership (6), verification_status (3), purpose (14), addr_state (50)

The list is taken from the numeric block printed by notebook 2 (Table 17 of the notebook) and the exact feature lists are stored in `feature_info.json`.